GGPLOT2 GALLERY

Contents

geom_abline 2
geom_area 8
geom_bar 13
geom_bin_2d 21
geom_bin2d 25
geom_blank 29
geom_boxplot 30
geom_col 39
geom_contour 47
geom_contour_filled 54

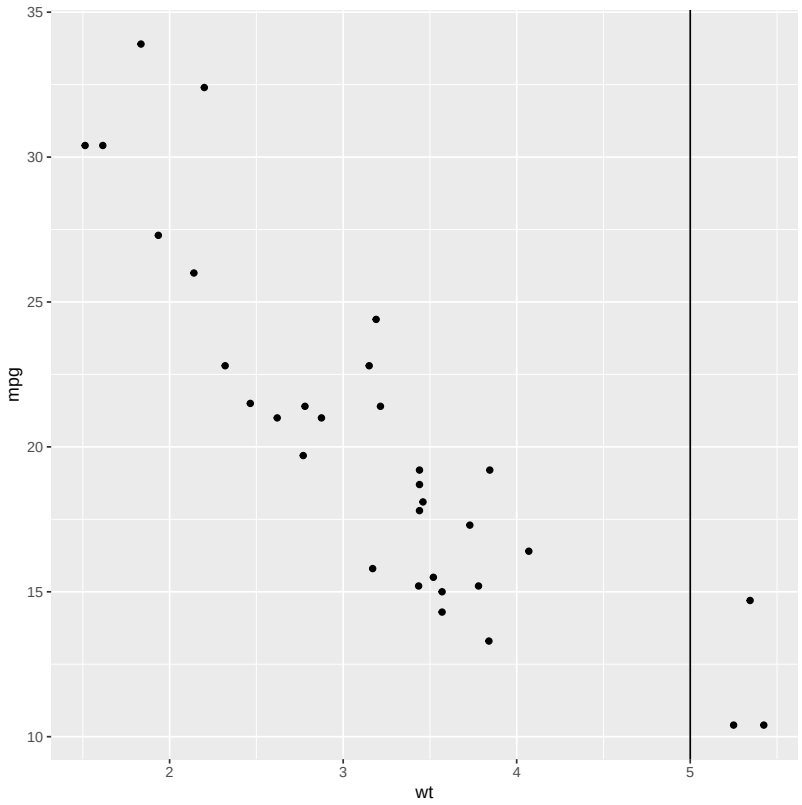
geom_abline

```
### Name: geom_abline
### Title: Reference lines: horizontal, vertical, and diagonal
### Aliases: geom_abline geom_hline geom_vline

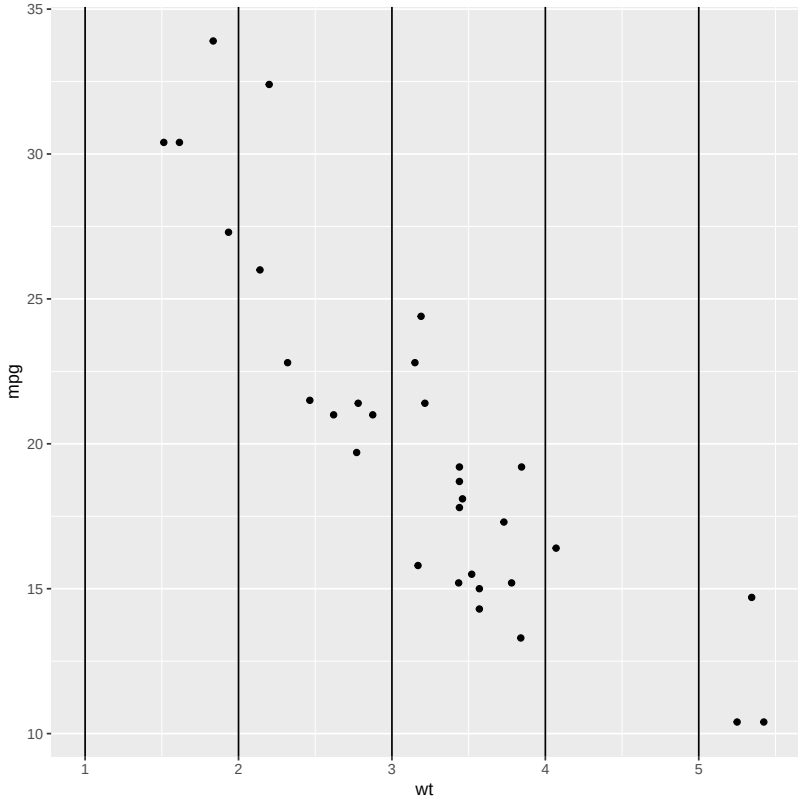
### ** Examples

p <- ggplot(mtcars, aes(wt, mpg)) + geom_point()

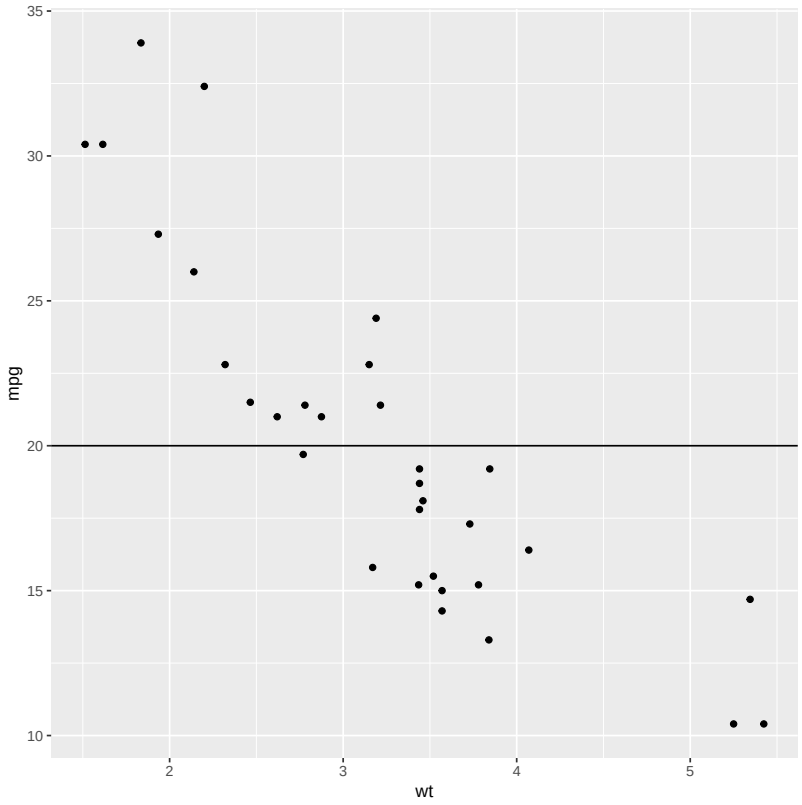
# Fixed values
p + geom_vline(xintercept = 5)
```



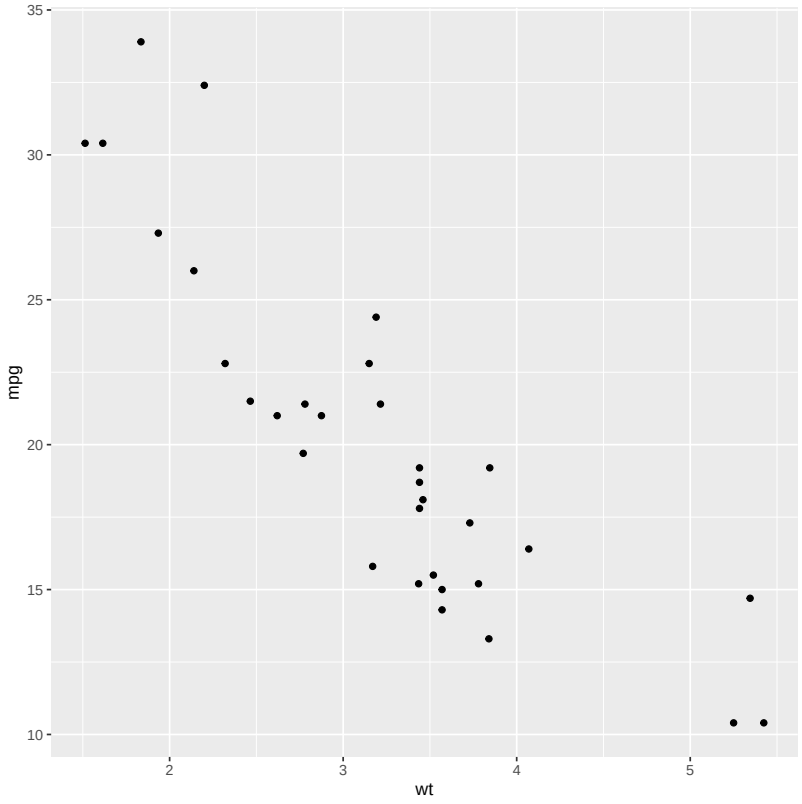

```
p + geom_vline(xintercept = 1:5)
```



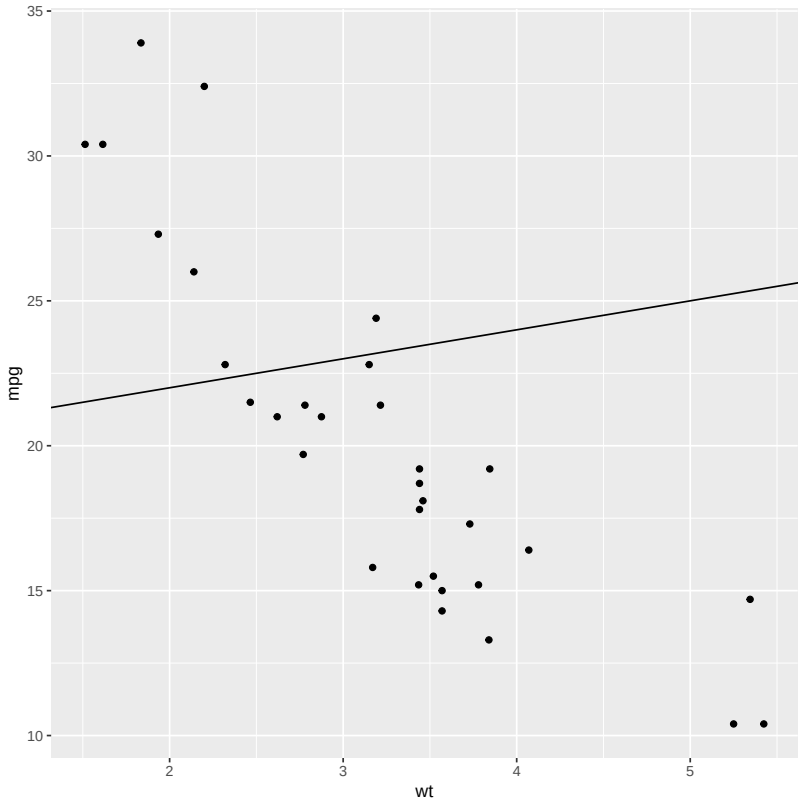
```
p + geom_hline(yintercept = 20)
```




```
p + geom_abline() # Can't see it - outside the range of the data
```



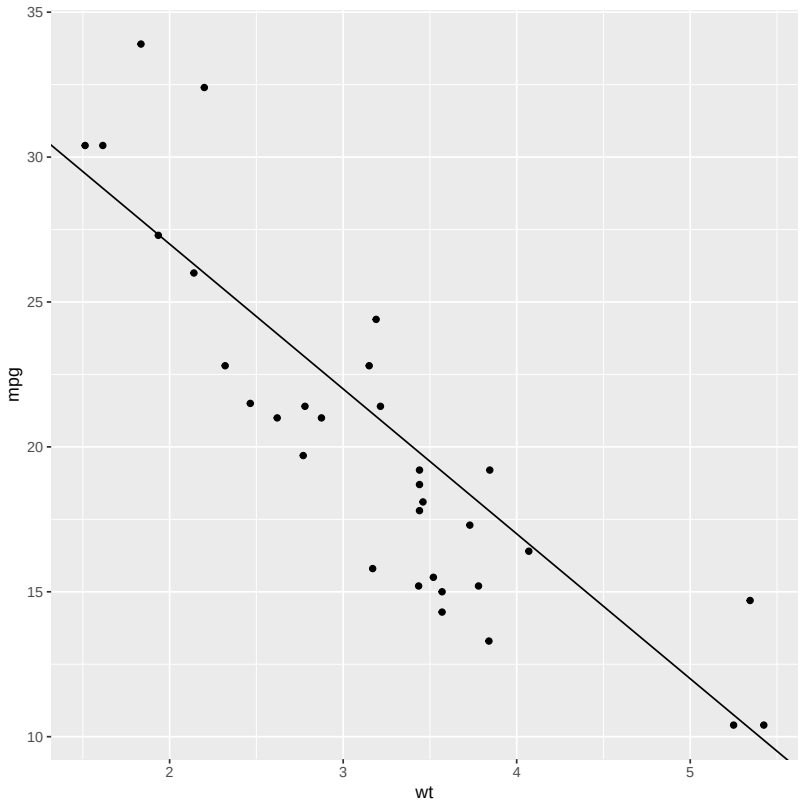
```
p + geom_abline(intercept = 20)
```




```
# Calculate slope and intercept of line of best fit
coef(lm(mpg ~ wt, data = mtcars))

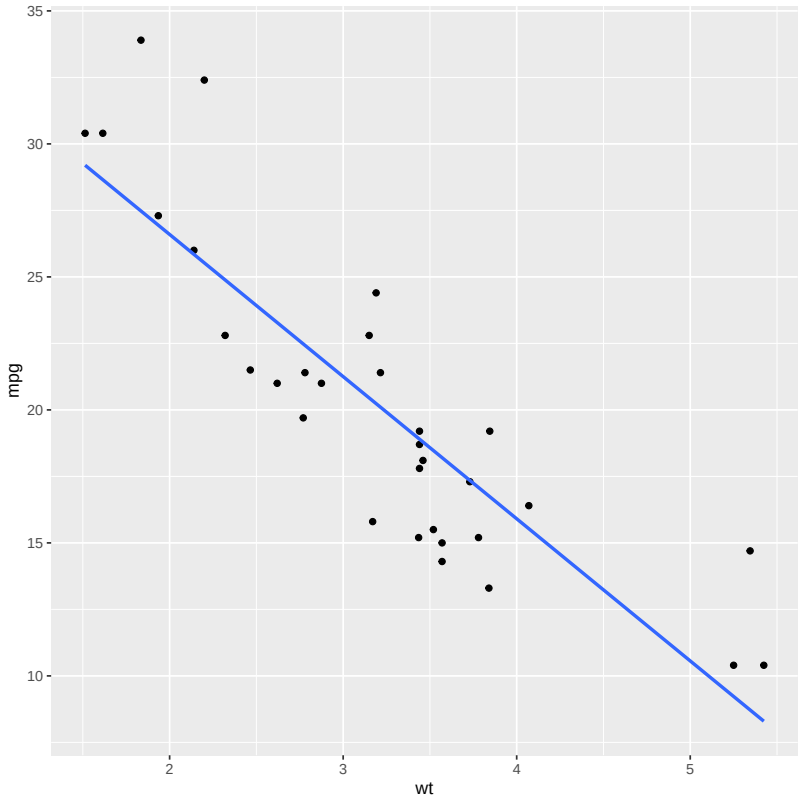
## (Intercept)          wt
##      37.285      -5.344

p + geom_abline(intercept = 37, slope = -5)
```



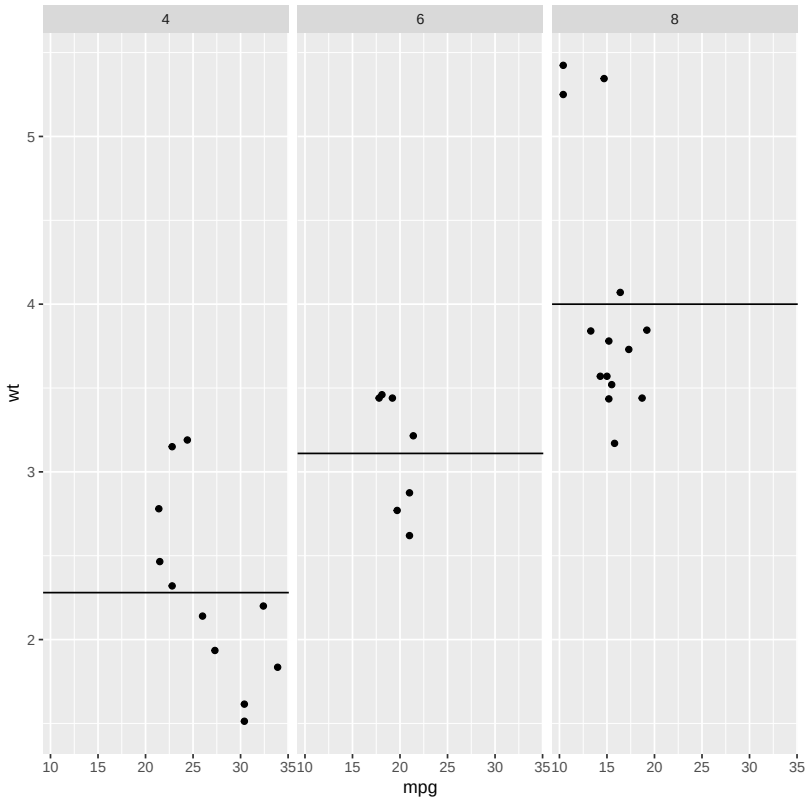

```
# But this is easier to do with geom_smooth:
p + geom_smooth(method = "lm", se = FALSE)

## `geom_smooth()` using formula = 'y ~ x'
```

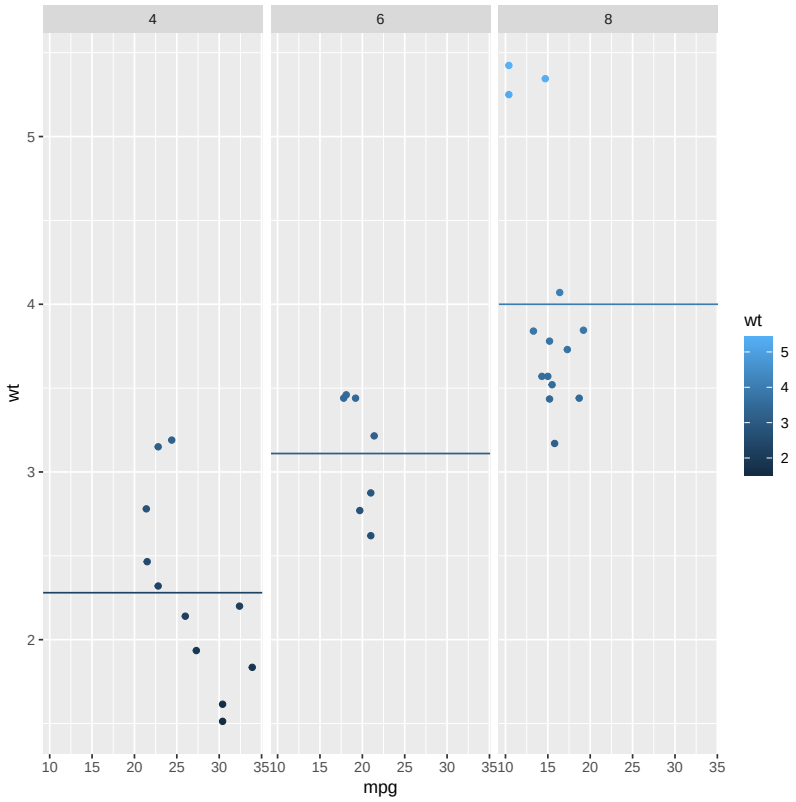


```
# To show different lines in different facets, use aesthetics
p <- ggplot(mtcars, aes(mpg, wt)) +
  geom_point() +
  facet_wrap(~ cyl)

mean_wt <- data.frame(cyl = c(4, 6, 8), wt = c(2.28, 3.11, 4.00))
p + geom_hline(aes(yintercept = wt), mean_wt)
```




```
# You can also control other aesthetics
ggplot(mtcars, aes(mpg, wt, colour = wt)) +
  geom_point() +
  geom_hline(aes(yintercept = wt, colour = wt), mean_wt) +
  facet_wrap(~ cyl)
```



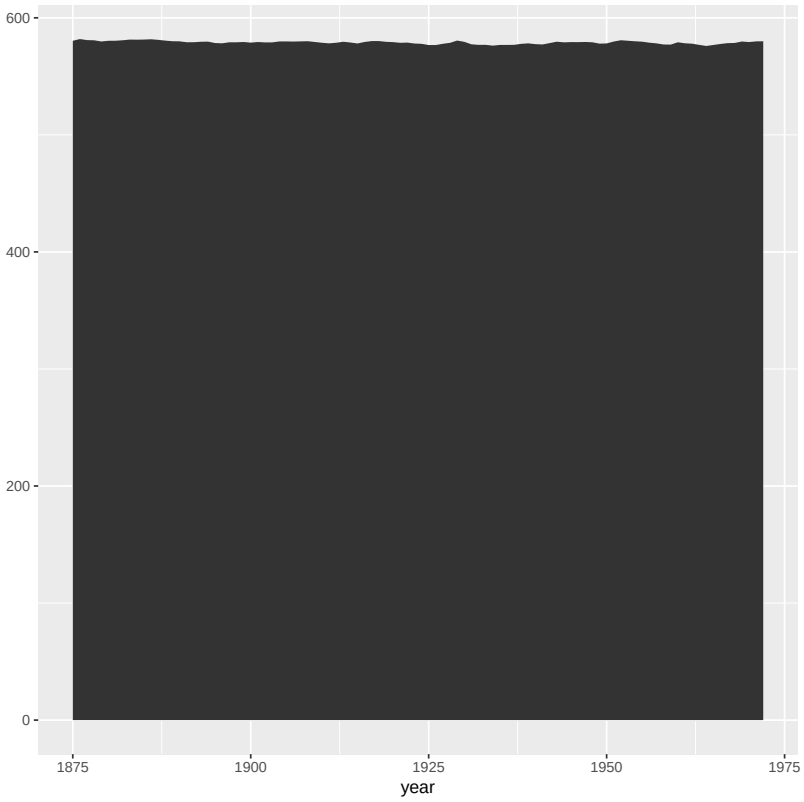
geom_area

```
### Name: geom_ribbon
### Title: Ribbons and area plots
### Aliases: geom_ribbon geom_area stat_align

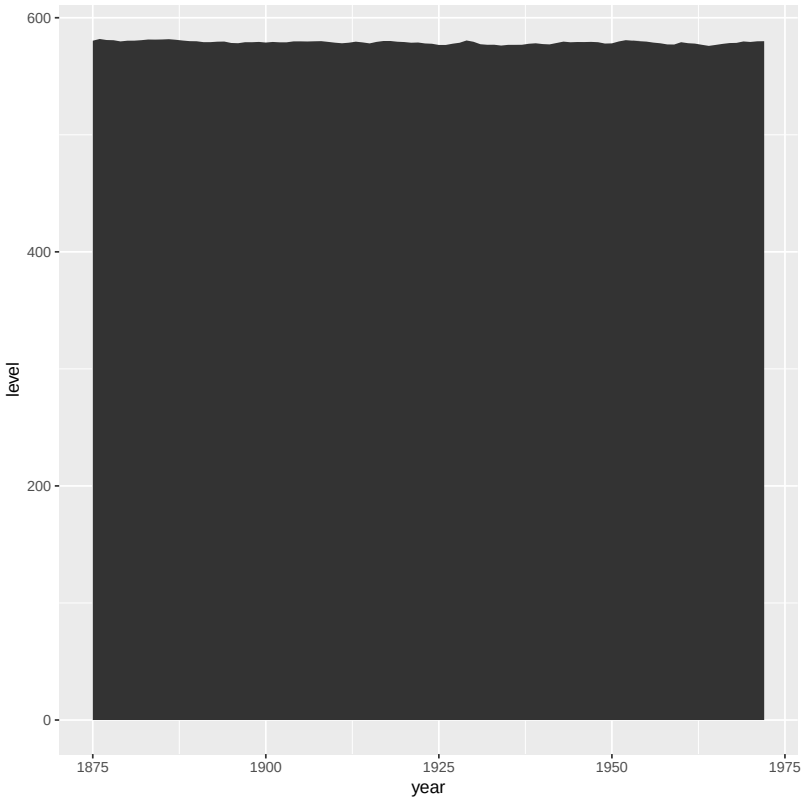
### ** Examples

# Generate data
huron <- data.frame(year = 1875:1972, level = as.vector(LakeHuron))
h <- ggplot(huron, aes(year))

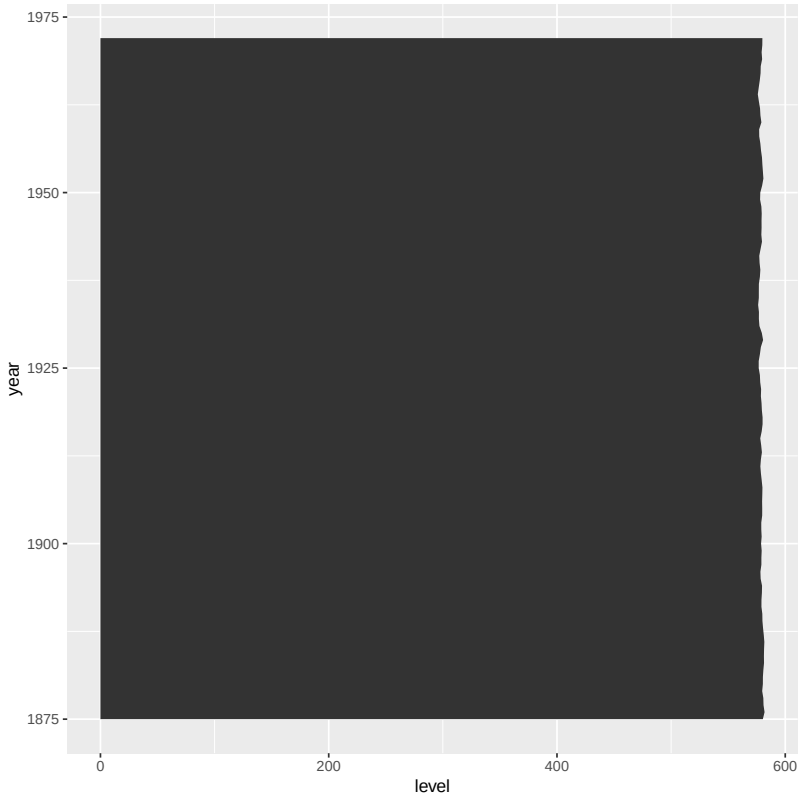
h + geom_ribbon(aes(ymin=0, ymax=level))
```



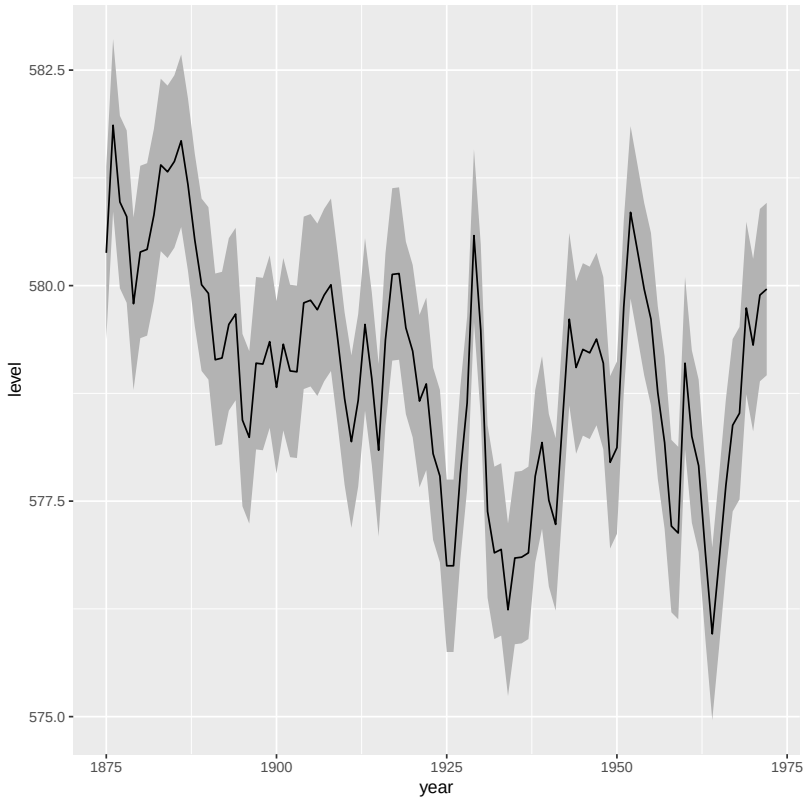

```
h + geom_area(aes(y = level))
```



```
# Orientation cannot be deduced by mapping, so must be given explicitly for
# flipped orientation
h + geom_area(aes(x = level, y = year), orientation = "y")
```

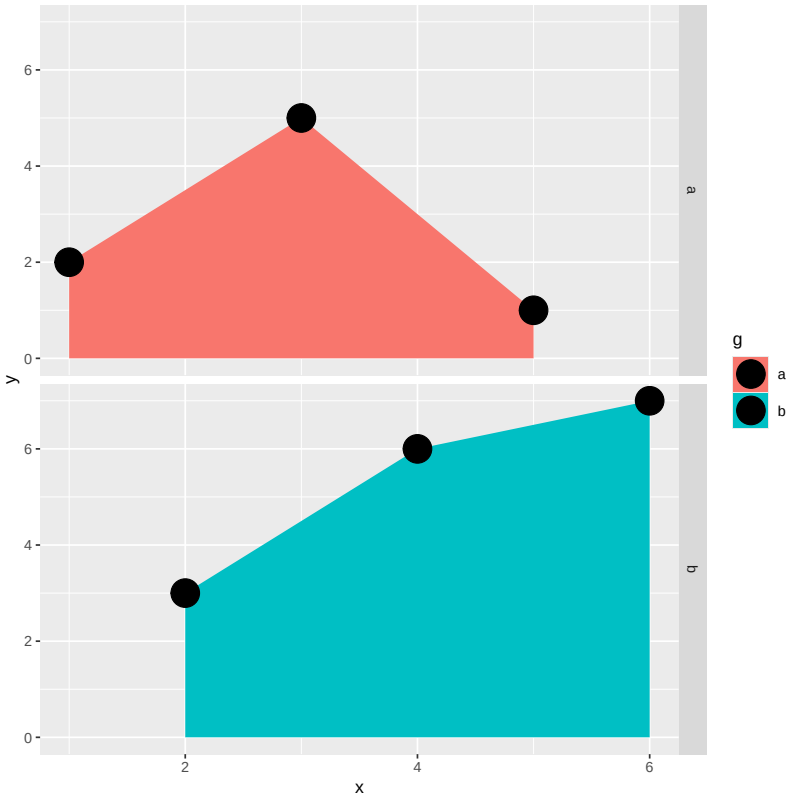



```
# Add aesthetic mappings
h +
  geom_ribbon(aes(ymin = level - 1, ymax = level + 1), fill = "grey70") +
  geom_line(aes(y = level))
```

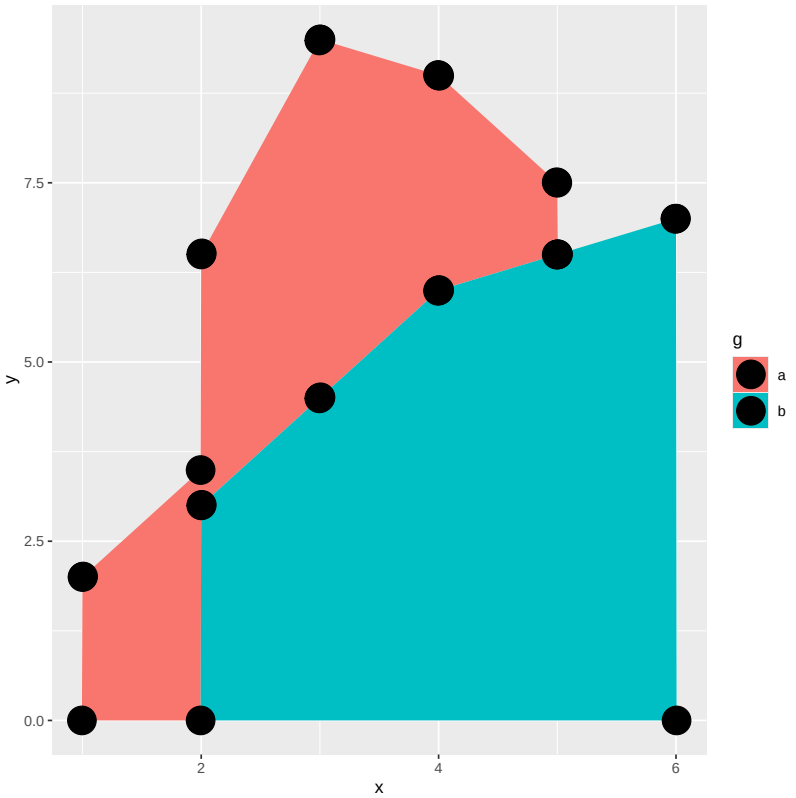



```r
# The underlying stat_align() takes care of unaligned data points
df <- data.frame(
  g = c("a", "a", "a", "b", "b", "b"),
  x = c(1, 3, 5, 2, 4, 6),
  y = c(2, 5, 1, 3, 6, 7)
)
a <- ggplot(df, aes(x, y, fill = g)) +
  geom_area()

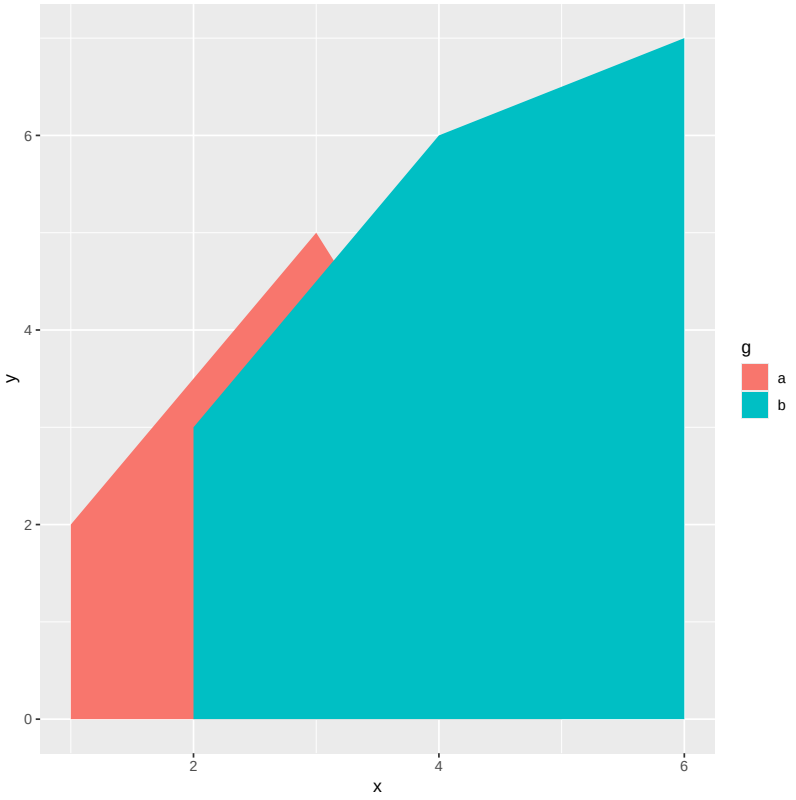
# Two groups have points on different X values.
a + geom_point(size = 8) + facet_grid(g ~ .)
```




```
# stat_align() interpolates and aligns the value so that the areas can stack
# properly.
a + geom_point(stat = "align", position = "stack", size = 8)
```



```
# To turn off the alignment, the stat can be set to "identity"
ggplot(df, aes(x, y, fill = g)) +
  geom_area(stat = "identity")
```

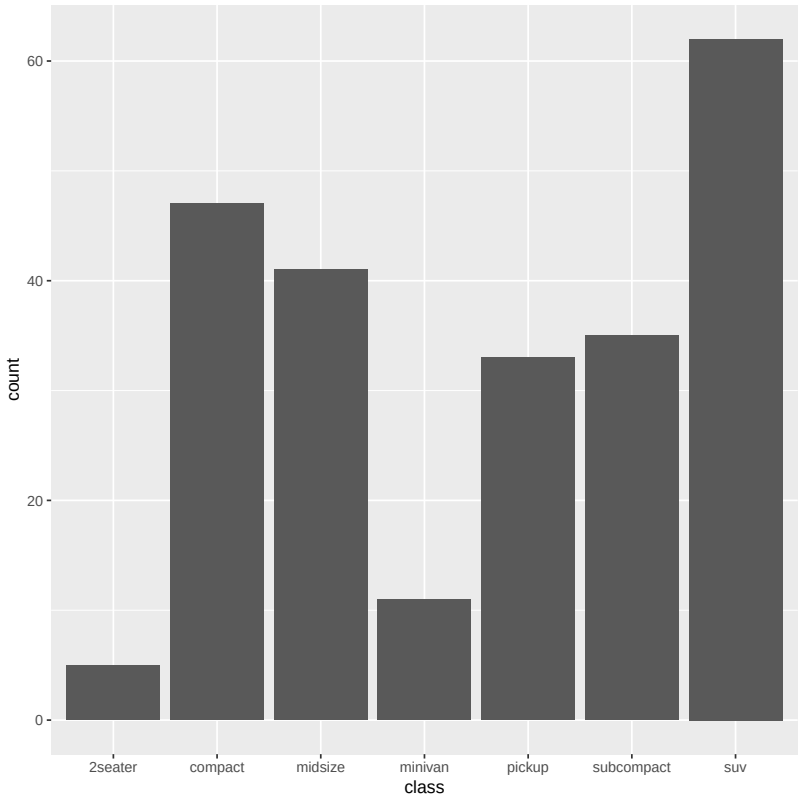


geom_bar

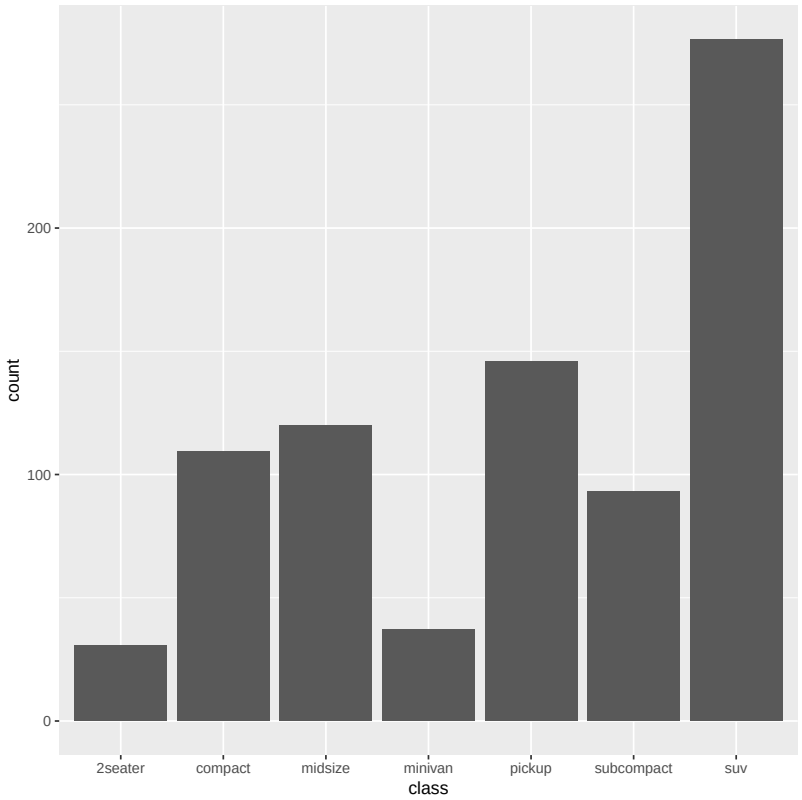
```
### Name: geom_bar
### Title: Bar charts
### Aliases: geom_bar geom_col stat_count

### ** Examples

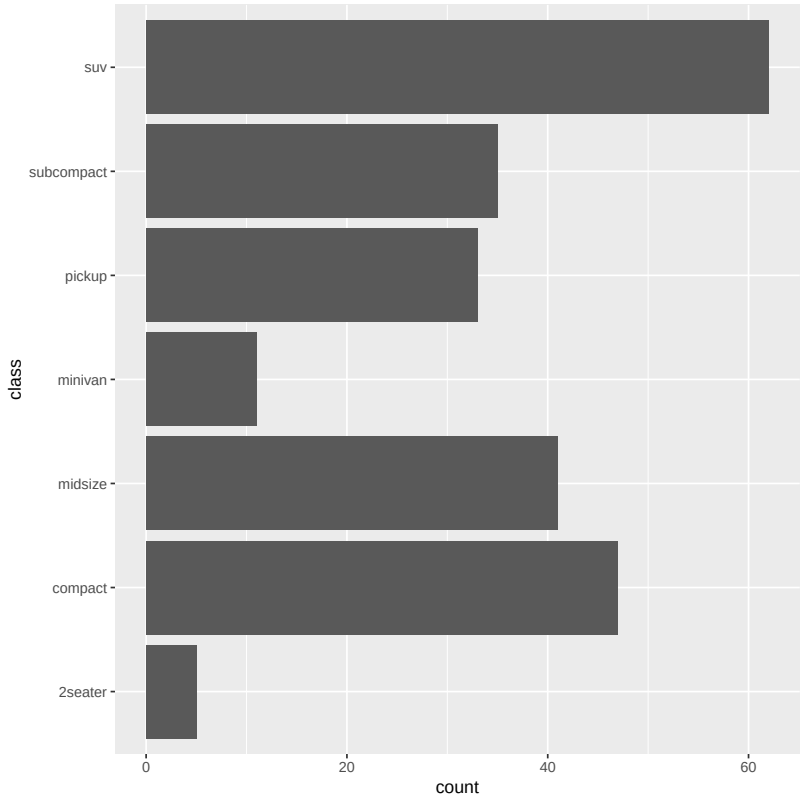
# geom_bar is designed to make it easy to create bar charts that show
# counts (or sums of weights)
g <- ggplot(mpg, aes(class))
# Number of cars in each class:
g + geom_bar()
```



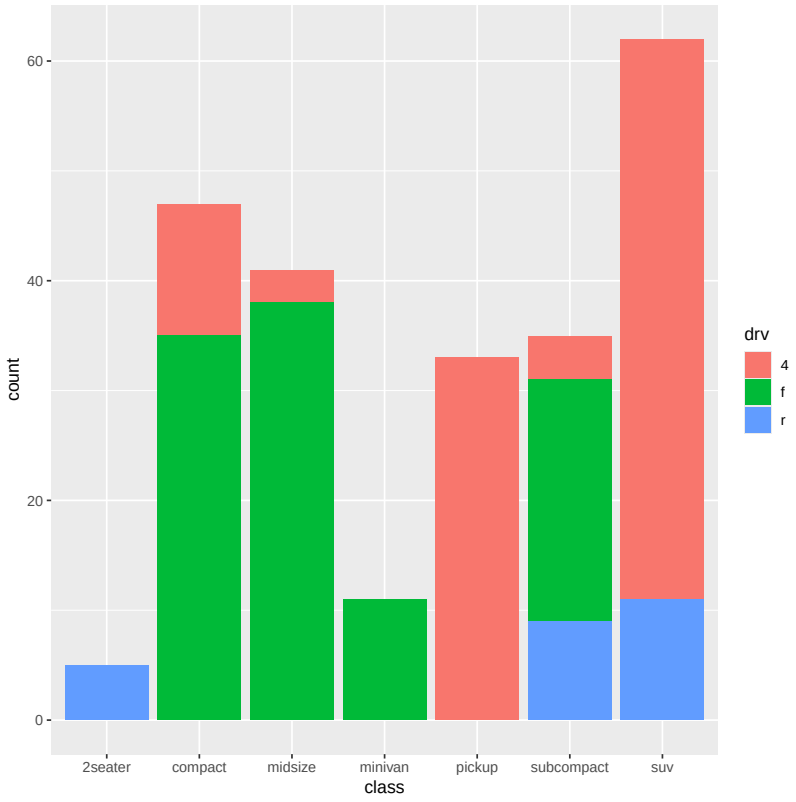

```
# Total engine displacement of each class
g + geom_bar(aes(weight = displ))
```



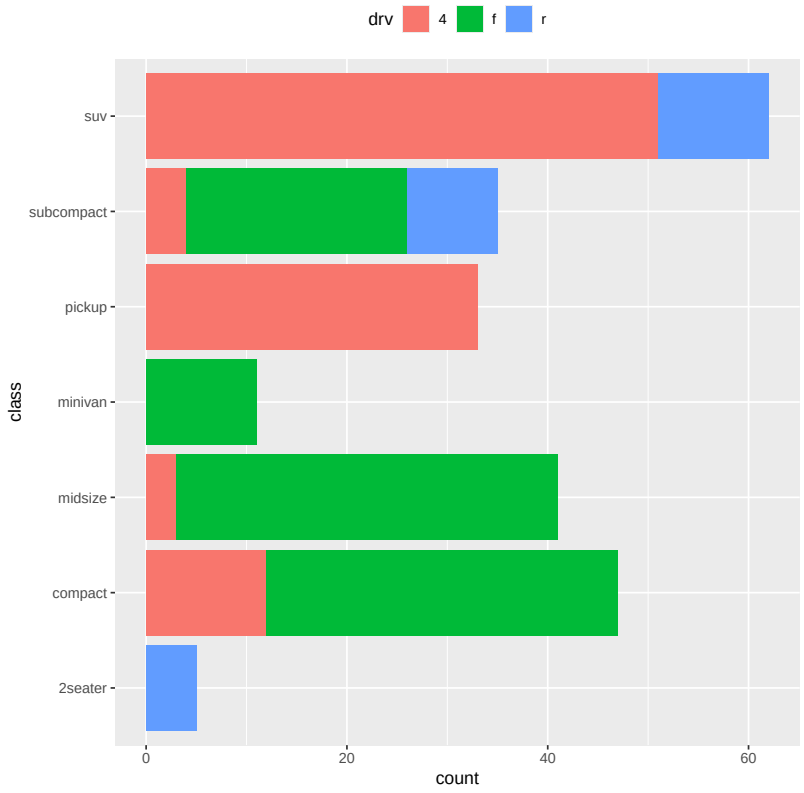
```
# Map class to y instead to flip the orientation
ggplot(mpg) + geom_bar(aes(y = class))
```



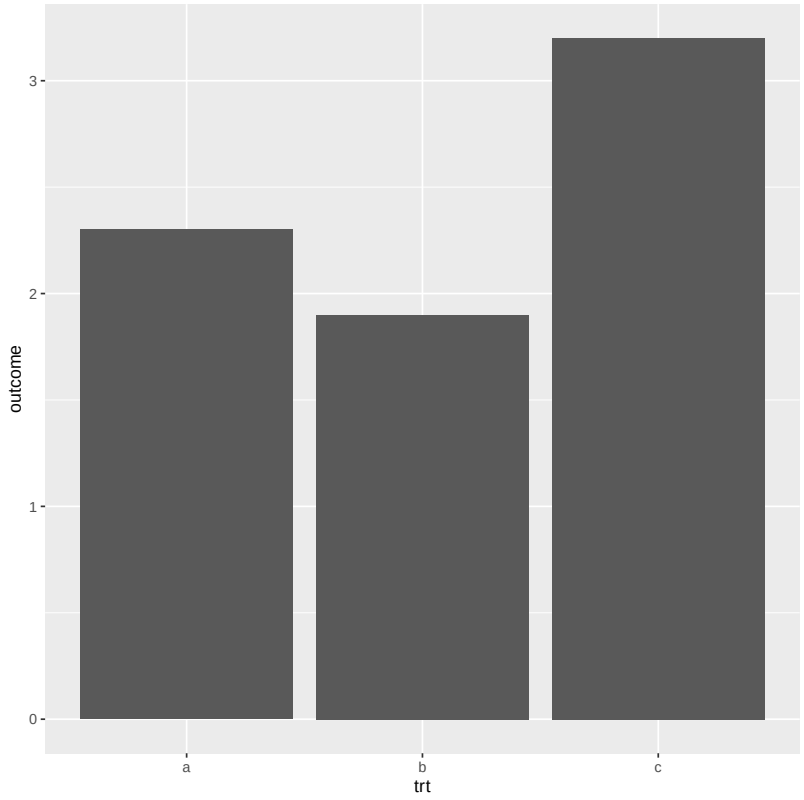

```
# Bar charts are automatically stacked when multiple bars are placed
# at the same location. The order of the fill is designed to match
# the legend
g + geom_bar(aes(fill = drv))
```



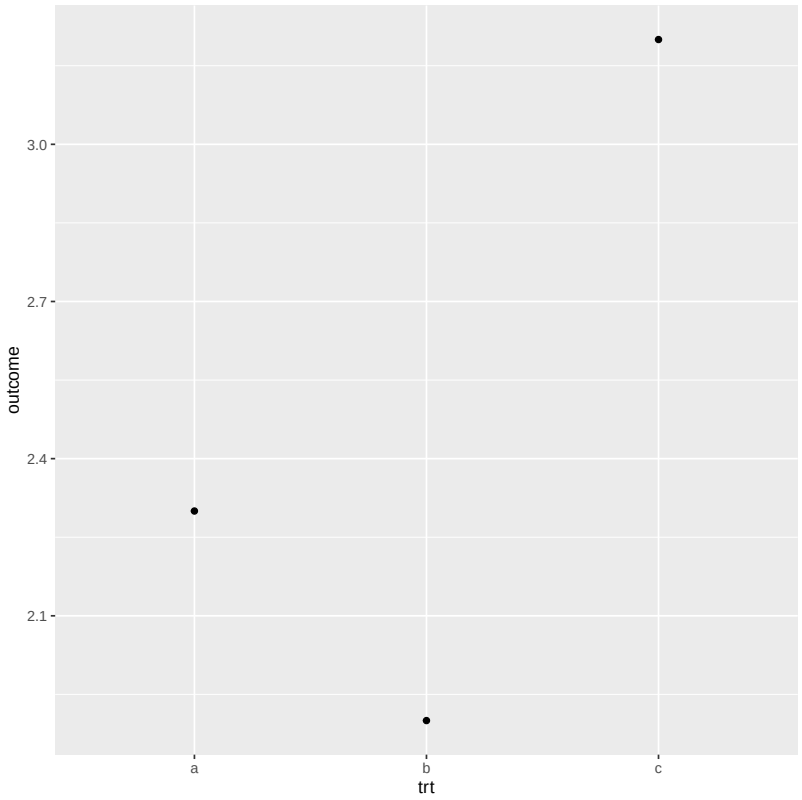

```
# If you need to flip the order (because you've flipped the orientation)
# call position_stack() explicitly:
ggplot(mpg, aes(y = class)) +
 geom_bar(aes(fill = drv), position = position_stack(reverse = TRUE)) +
 theme(legend.position = "top")
```



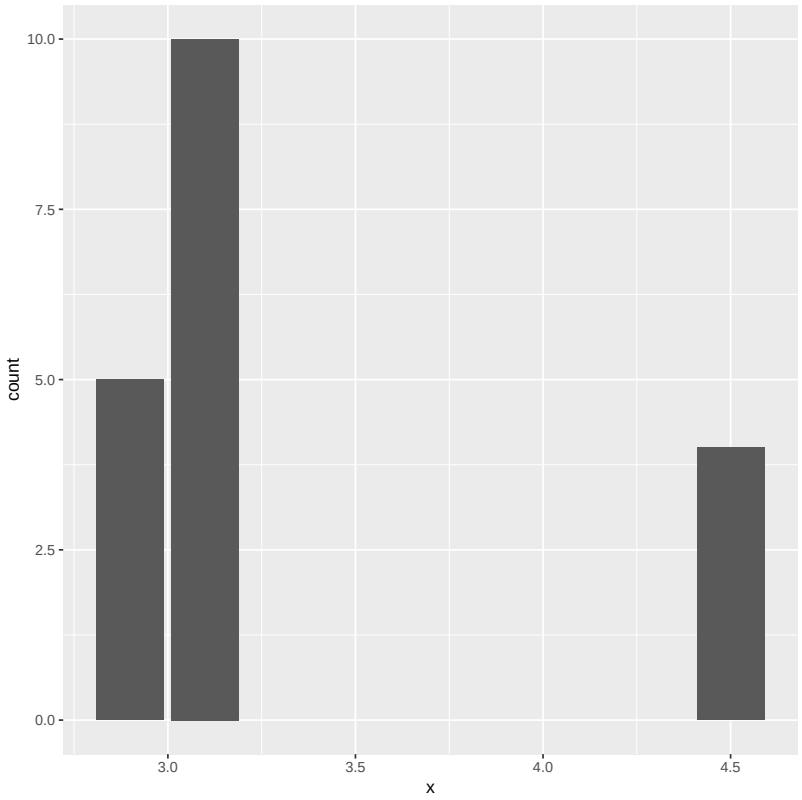

```
# To show (e.g.) means, you need geom_col()
df <- data.frame(trt = c("a", "b", "c"), outcome = c(2.3, 1.9, 3.2))
ggplot(df, aes(trt, outcome)) +
  geom_col()
```



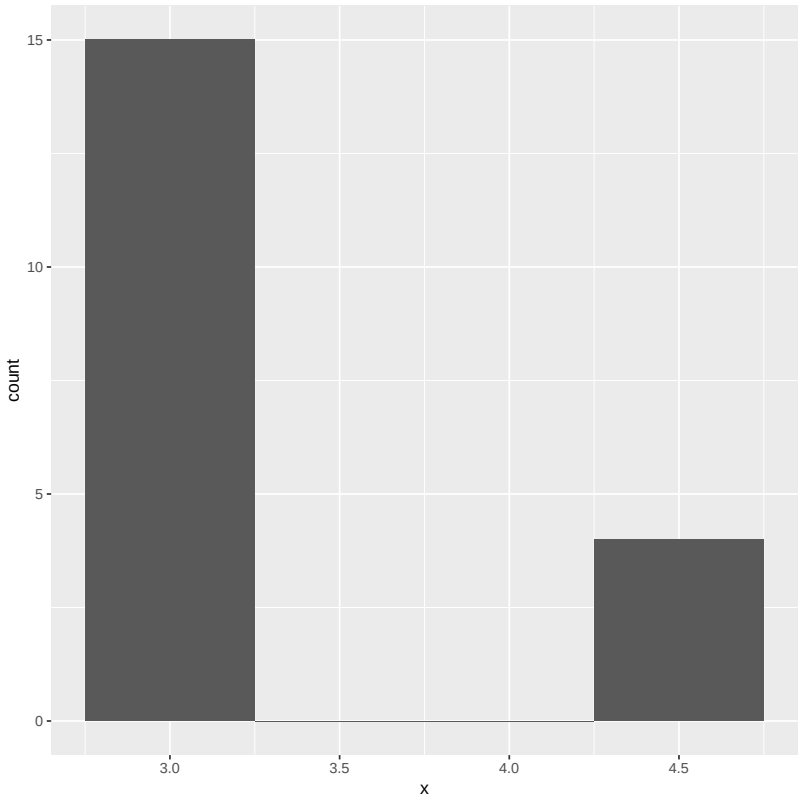
```
# But geom_point() displays exactly the same information and doesn't
# require the y-axis to touch zero.
ggplot(df, aes(trt, outcome)) +
  geom_point()
```



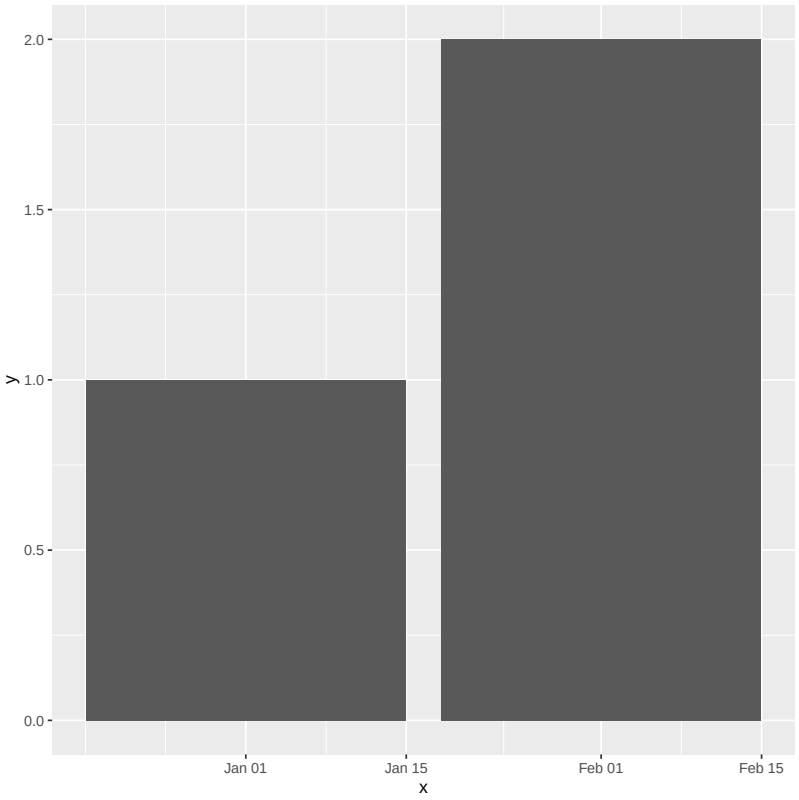

```
# You can also use geom_bar() with continuous data, in which case
# it will show counts at unique locations
df <- data.frame(x = rep(c(2.9, 3.1, 4.5), c(5, 10, 4)))
ggplot(df, aes(x)) + geom_bar()
```



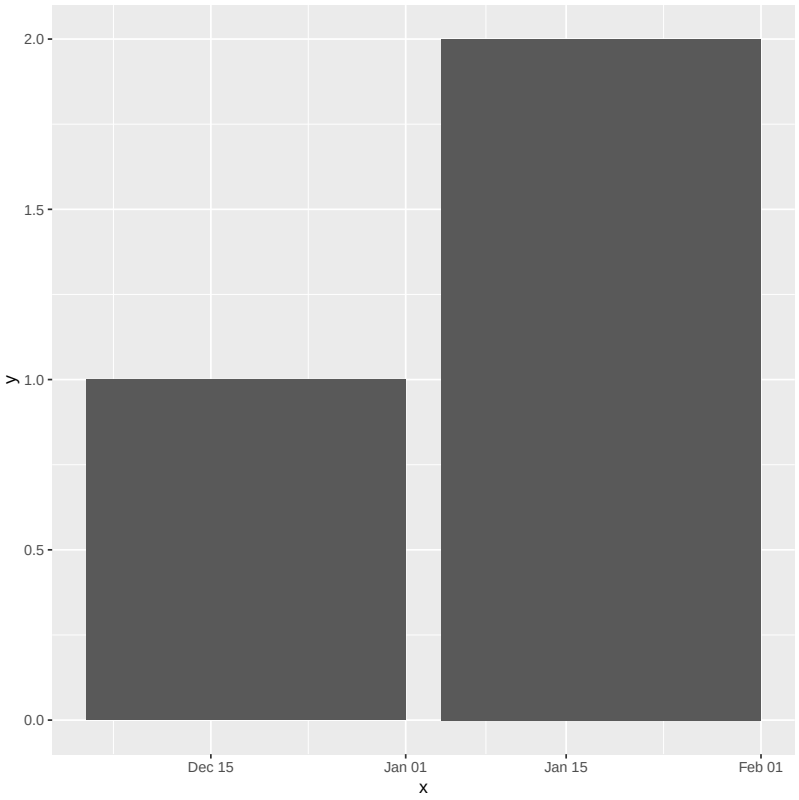

```
# cf. a histogram of the same data
ggplot(df, aes(x)) + geom_histogram(binwidth = 0.5)
```



```r
# Use `just` to control how columns are aligned with axis breaks:
df <- data.frame(x = as.Date(c("2020-01-01", "2020-02-01")), y = 1:2)
# Columns centered on the first day of the month
ggplot(df, aes(x, y)) + geom_col(just = 0.5)
```




```r
# Columns begin on the first day of the month
ggplot(df, aes(x, y)) + geom_col(just = 1)
```



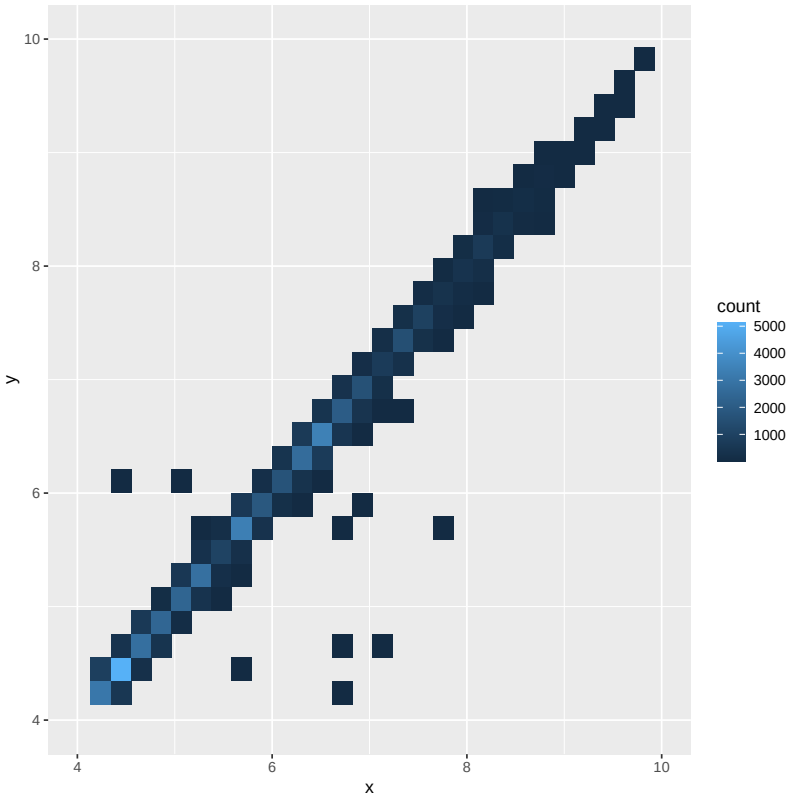
geom_bin_2d

```
### Name: geom_bin_2d
### Title: Heatmap of 2d bin counts
### Aliases: geom_bin_2d geom_bin2d stat_bin_2d stat_bin2d

### ** Examples

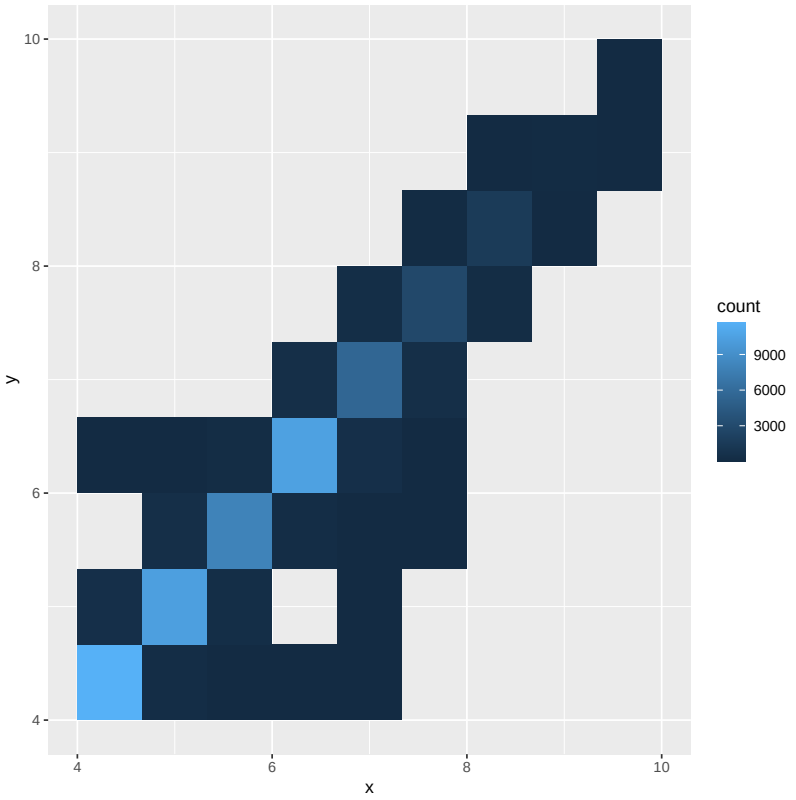
d <- ggplot(diamonds, aes(x, y)) + xlim(4, 10) + ylim(4, 10)
d + geom_bin_2d()

## `stat_bin2d()` using `bins = 30`.  Pick better value
`binwidth`.
## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (`stat_bin2d()`).
## Warning:  Removed 5 rows containing missing values or
values outside the scale range
## (`geom_bin2d()`).
```



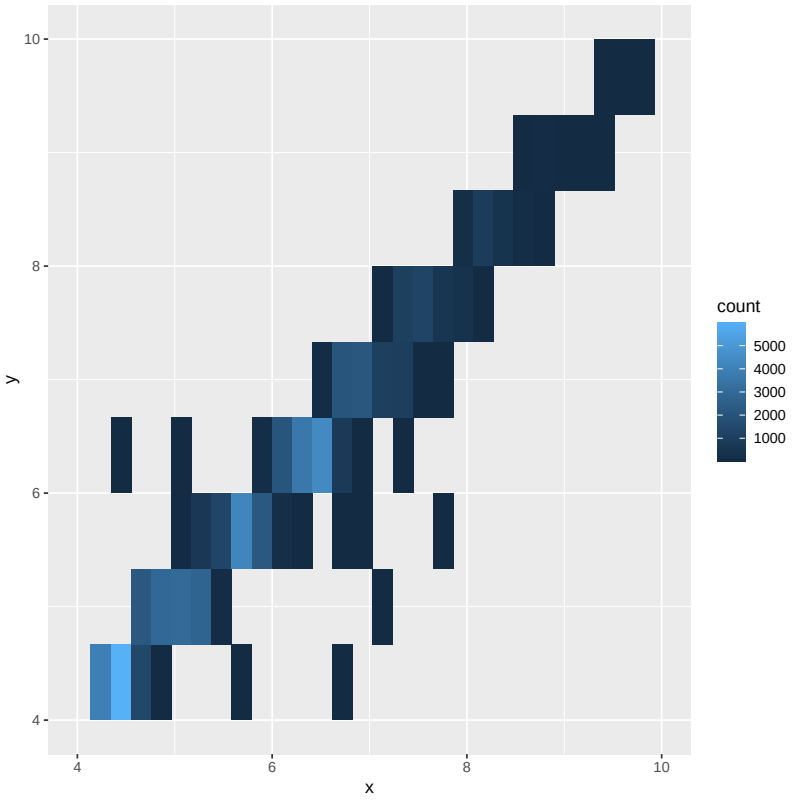

```
# You can control the size of the bins by specifying the number of
# bins in each direction:
d + geom_bin_2d(bins = 10)

## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
```



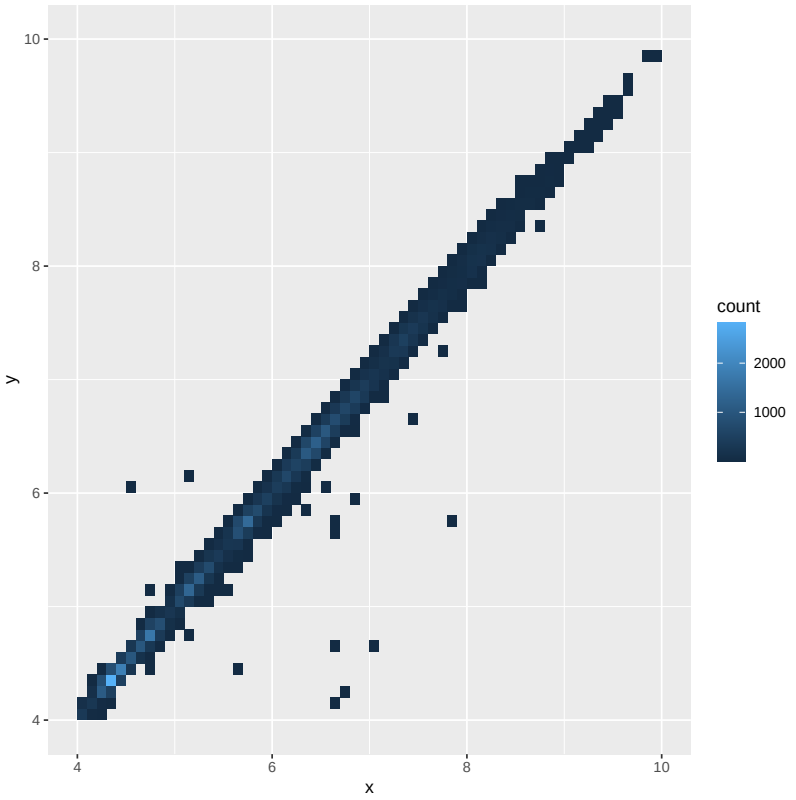

```
d + geom_bin_2d(bins = list(x = 30, y = 10))

## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
## Warning:  Removed 2 rows containing missing values or
values outside the scale range
## (‘geom_bin2d()‘).
```




```
# Or by specifying the width of the bins
d + geom_bin_2d(binwidth = c(0.1, 0.1))

## Warning: Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
```



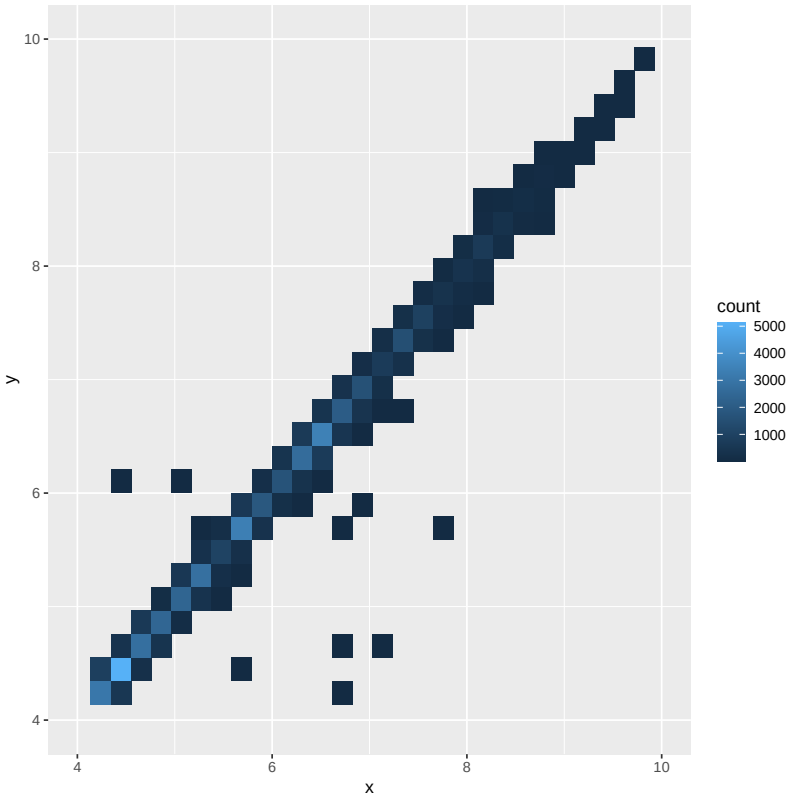
geom_bin2d

```
### Name: geom_bin_2d
### Title: Heatmap of 2d bin counts
### Aliases: geom_bin_2d geom_bin2d stat_bin_2d stat_bin2d

### ** Examples

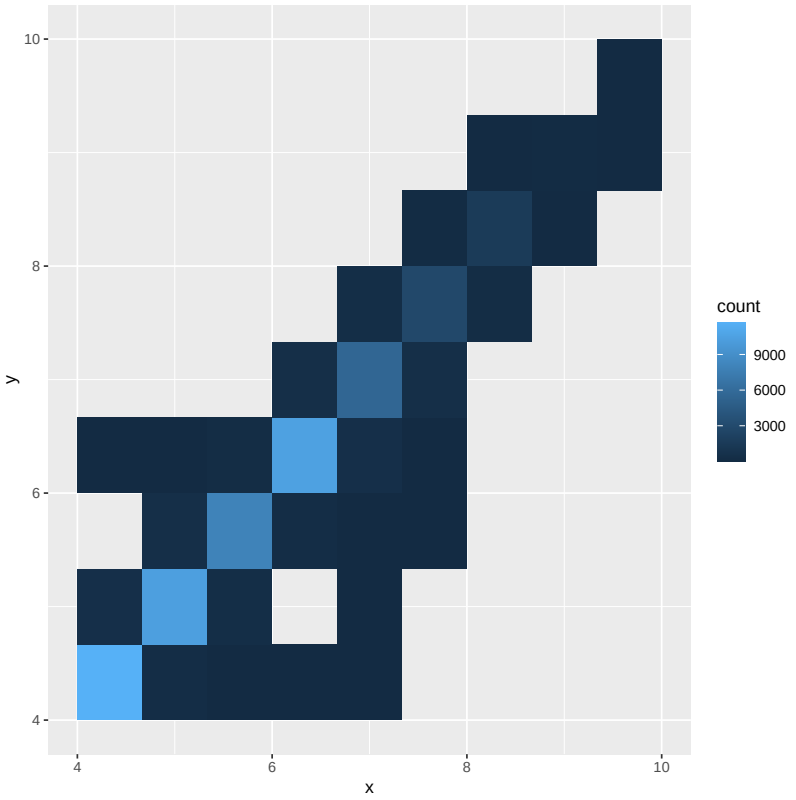
d <- ggplot(diamonds, aes(x, y)) + xlim(4, 10) + ylim(4, 10)
d + geom_bin_2d()

## `stat_bin2d()` using `bins = 30`.  Pick better value
`binwidth`.
## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (`stat_bin2d()`).
## Warning:  Removed 5 rows containing missing values or
values outside the scale range
## (`geom_bin2d()`).
```



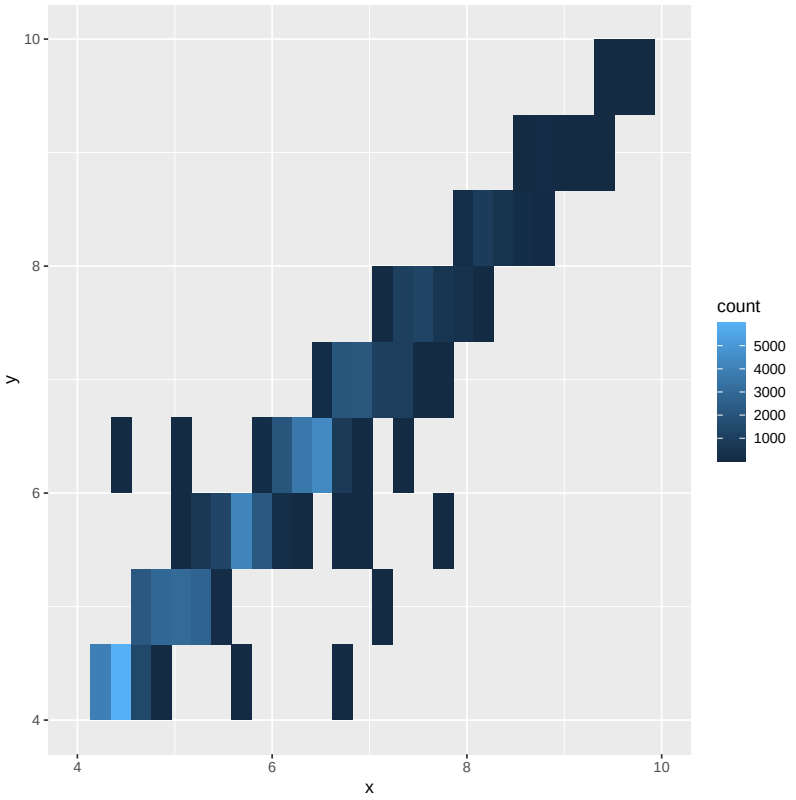

```
# You can control the size of the bins by specifying the number of
# bins in each direction:
d + geom_bin_2d(bins = 10)

## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
```



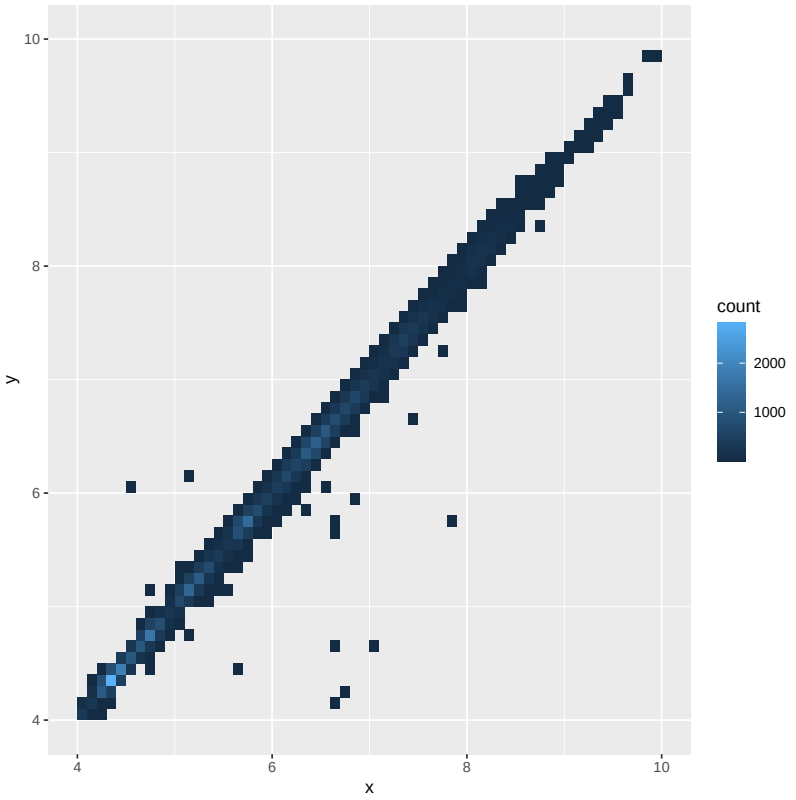

```
d + geom_bin_2d(bins = list(x = 30, y = 10))

## Warning:  Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
## Warning:  Removed 2 rows containing missing values or
values outside the scale range
## (‘geom_bin2d()‘).
```




```
# Or by specifying the width of the bins
d + geom_bin_2d(binwidth = c(0.1, 0.1))

## Warning: Removed 478 rows containing non-finite outside
the scale range
## (‘stat_bin2d()‘).
```

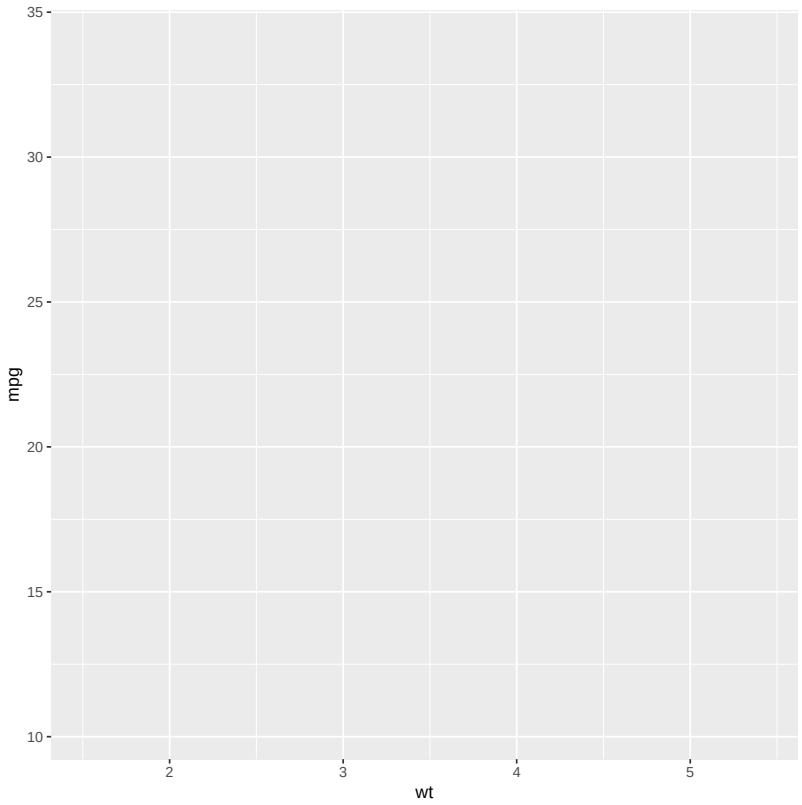


geom_blank

```
### Name: geom_blank
### Title: Draw nothing
### Aliases: geom_blank

### ** Examples

ggplot(mtcars, aes(wt, mpg))
```



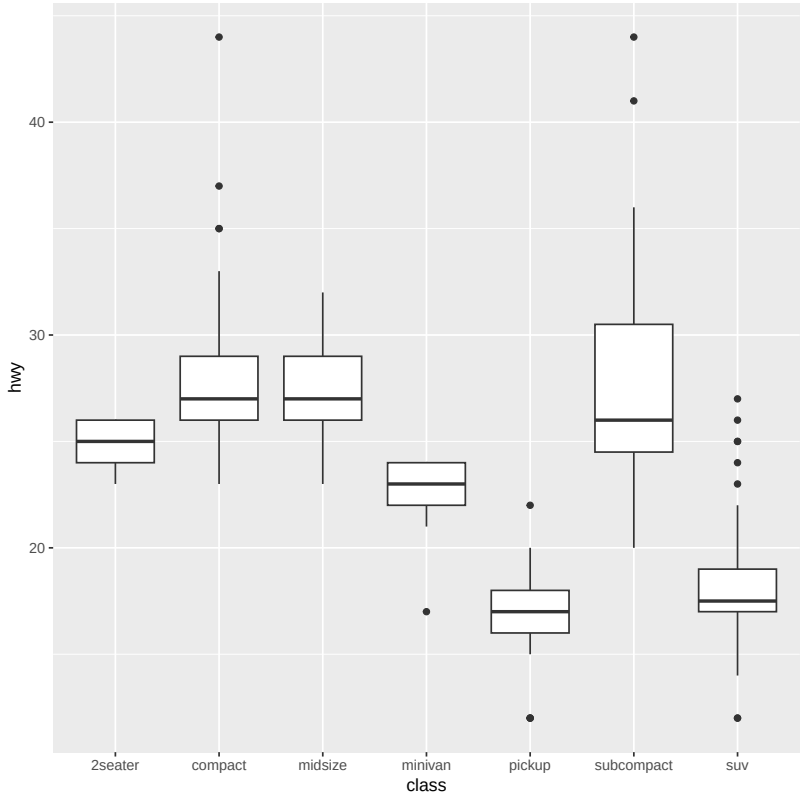

```
# Nothing to see here!
```

geom_boxplot

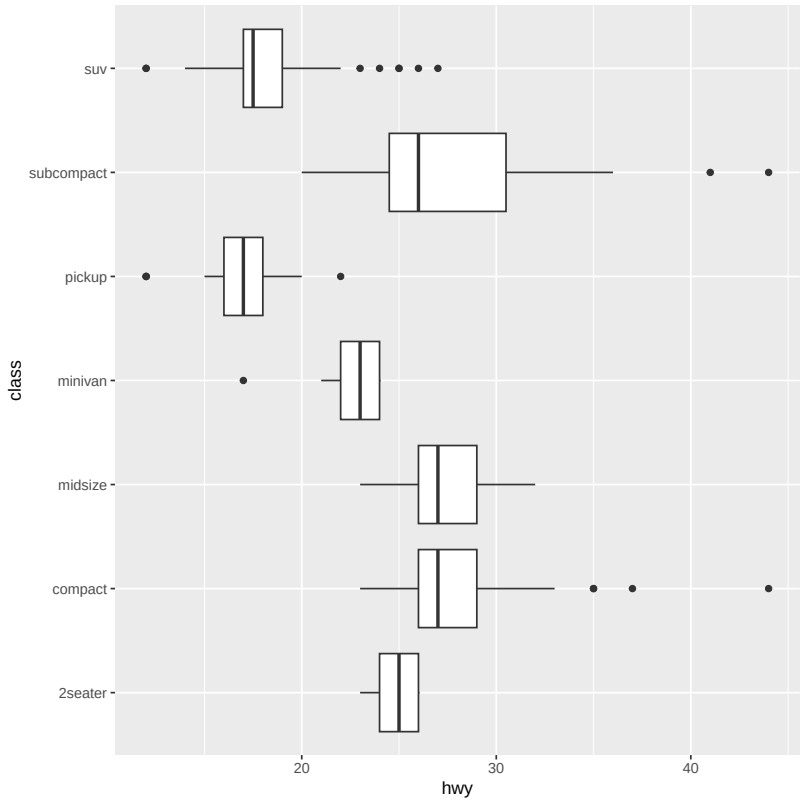
```
### Name: geom_boxplot
### Title: A box and whiskers plot (in the style of Tukey)
### Aliases: geom_boxplot stat_boxplot

### ** Examples

p <- ggplot(mpg, aes(class, hwy))
p + geom_boxplot()
```

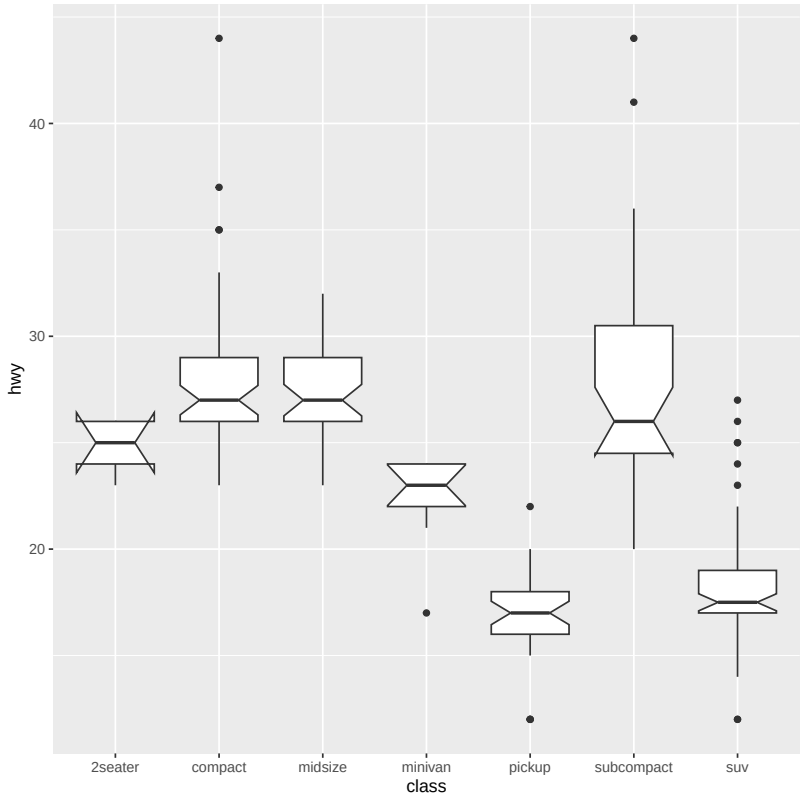



```
# Orientation follows the discrete axis
ggplot(mpg, aes(hwy, class)) + geom_boxplot()
```

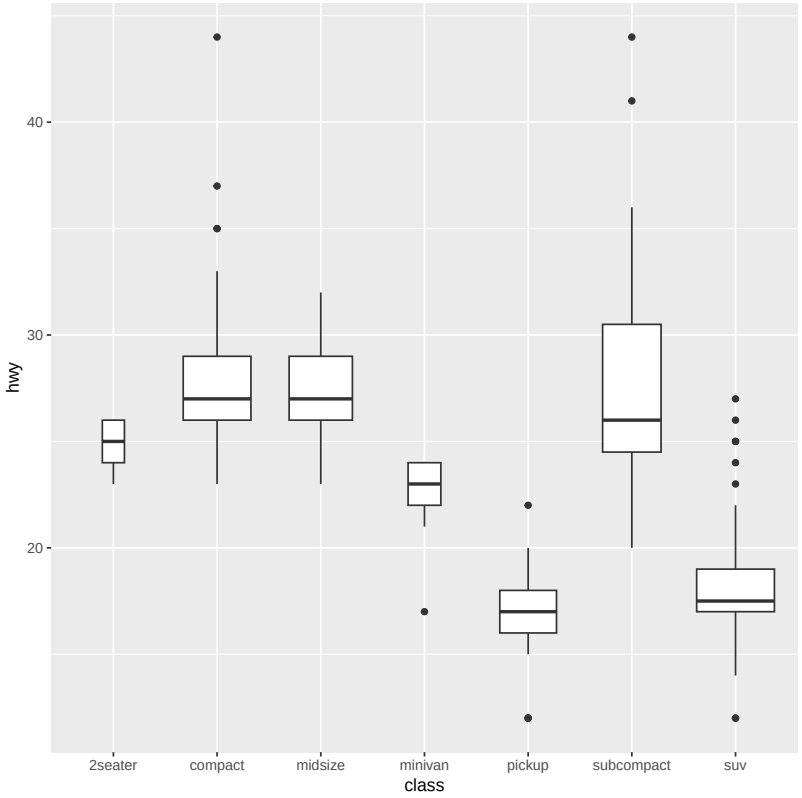


```
p + geom_boxplot(notch = TRUE)

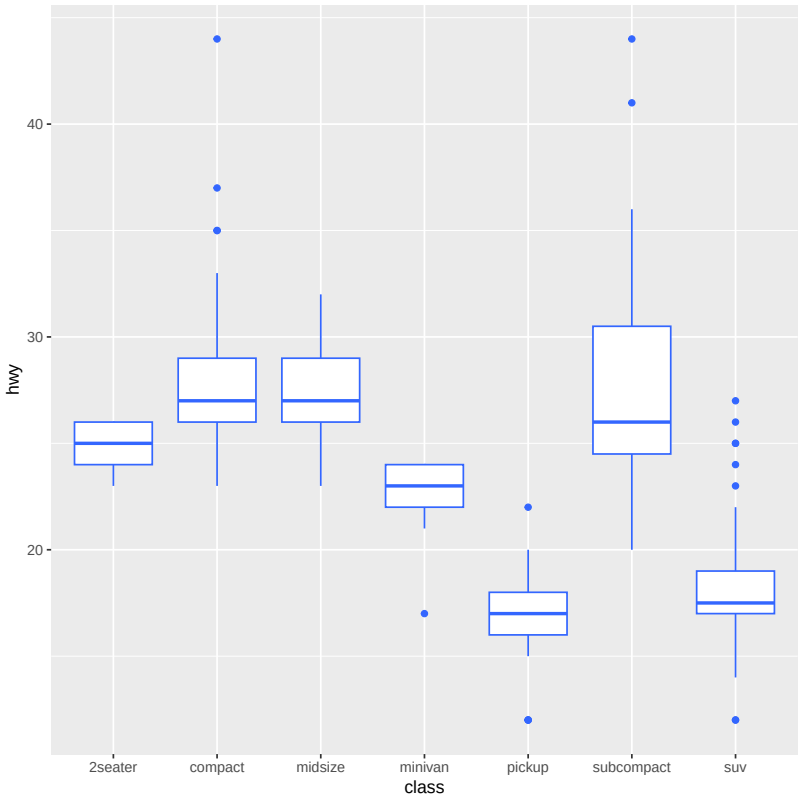
## Notch went outside hinges
## i Do you want `notch = FALSE`?
## Notch went outside hinges
## i Do you want `notch = FALSE`?
```



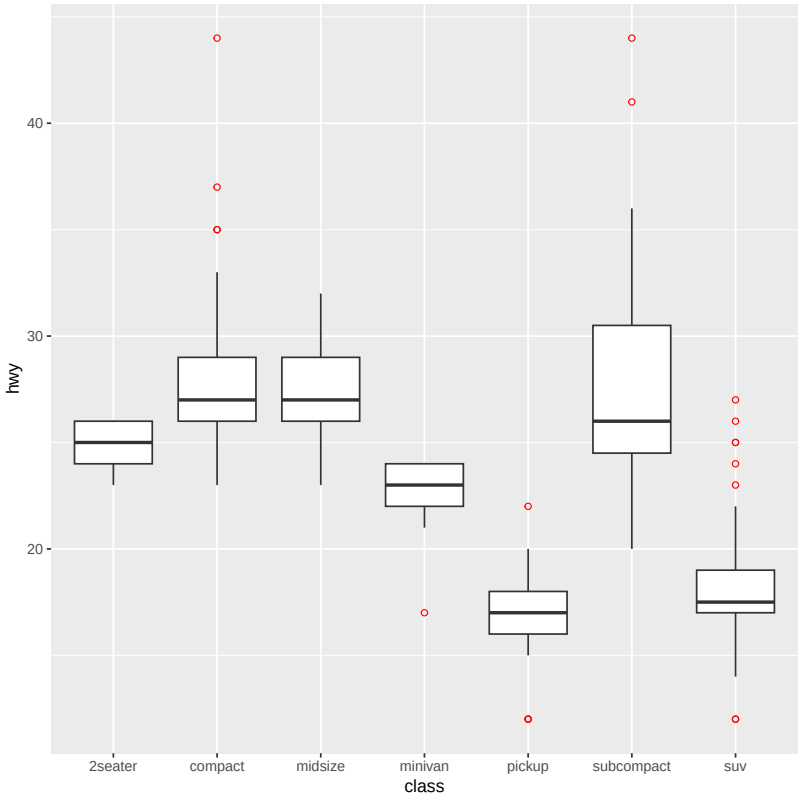

```
p + geom_boxplot(varwidth = TRUE)
```



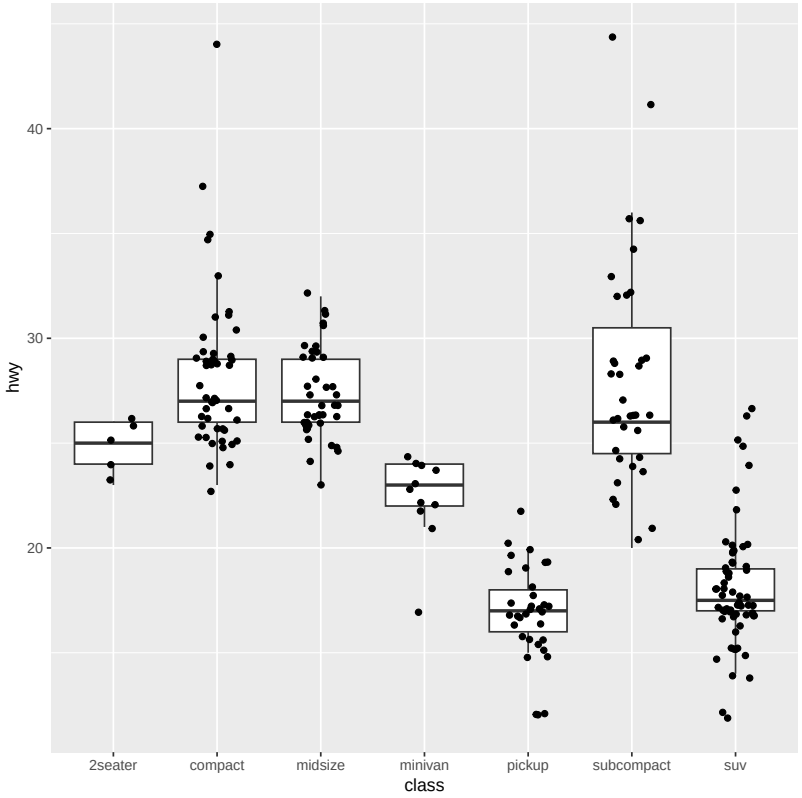

```
p + geom_boxplot(fill = "white", colour = "#3366FF")
```



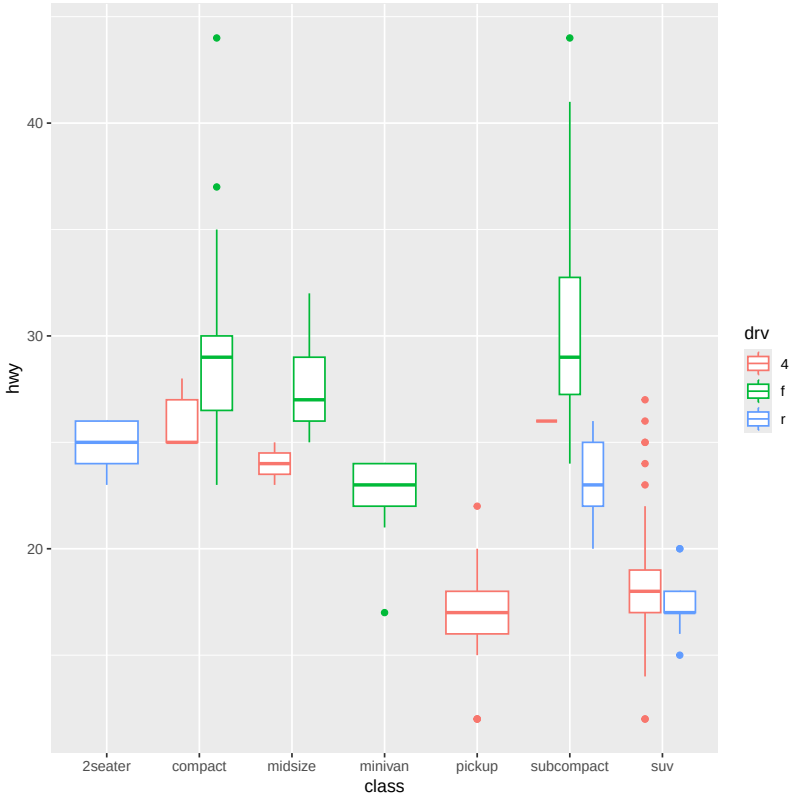
```
# By default, outlier points match the colour of the box. Use
# outlier.colour to override
p + geom_boxplot(outlier.colour = "red", outlier.shape = 1)
```




```
# Remove outliers when overlaying boxplot with original data points
p + geom_boxplot(outlier.shape = NA) + geom_jitter(width = 0.2)
```

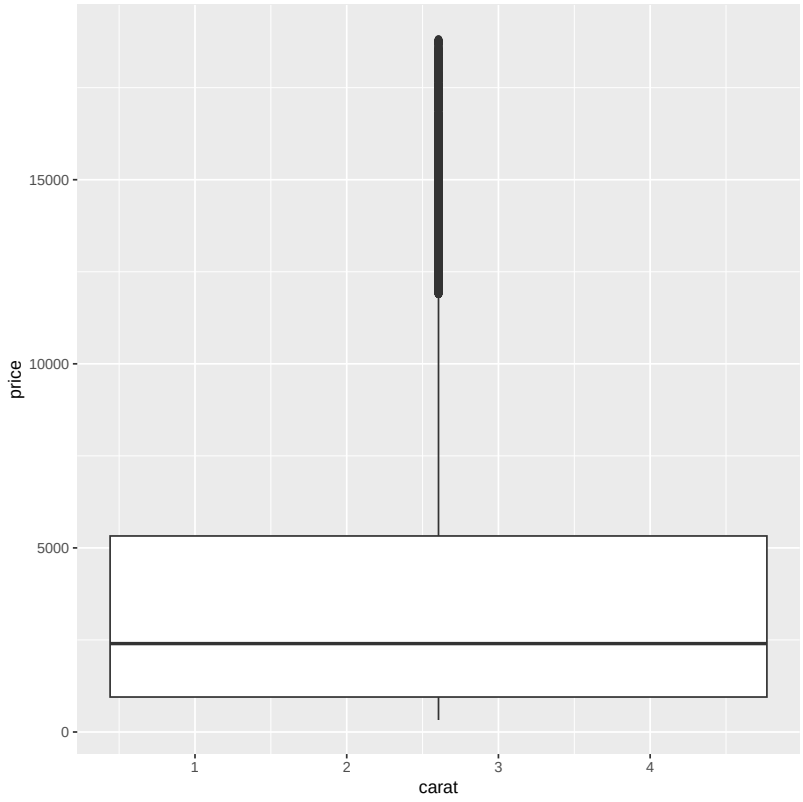



```
# Boxplots are automatically dodged when any aesthetic is a factor
p + geom_boxplot(aes(colour = drv))
```



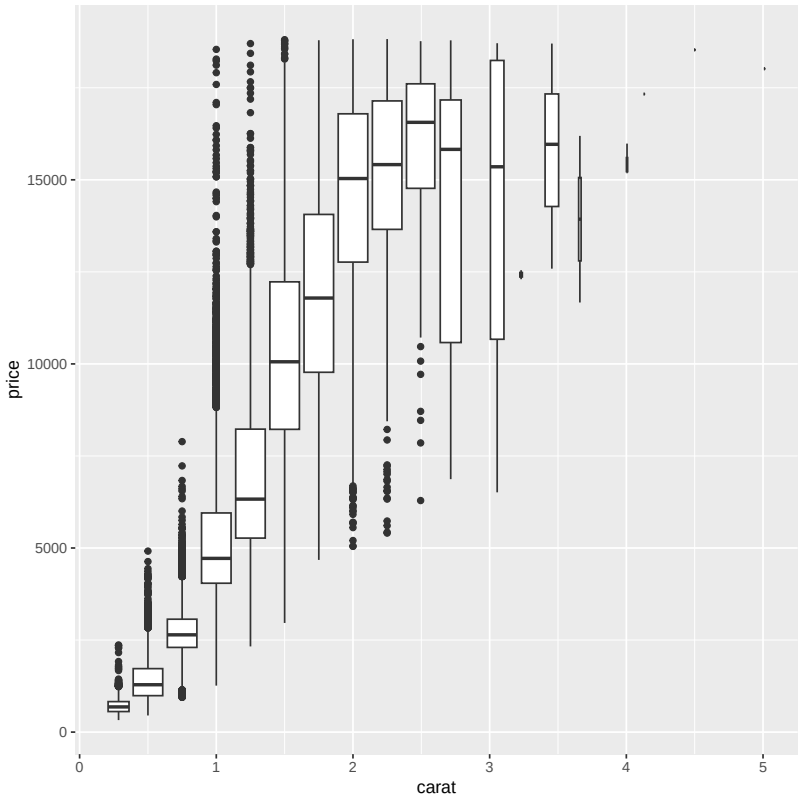
```
# You can also use boxplots with continuous x, as long as you supply
# a grouping variable. cut_width is particularly useful
ggplot(diamonds, aes(carat, price)) +
  geom_boxplot()

## Warning:  Orientation is not uniquely specified when
both the x and y aesthetics are
## continuous.  Picking default orientation 'x'.
## Warning:  Continuous x aesthetic
## i did you forget `aes(group = ...)`?
```



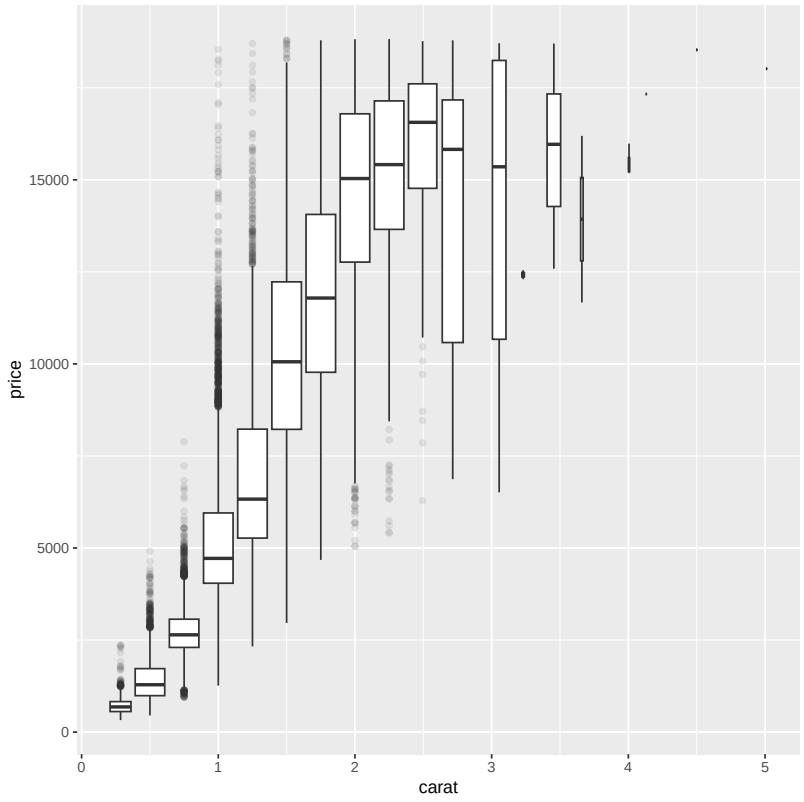

```
ggplot(diamonds, aes(carat, price)) +
  geom_boxplot(aes(group = cut_width(carat, 0.25)))

## Warning:  Orientation is not uniquely specified when
both the x and y aesthetics are
## continuous.  Picking default orientation 'x'.
```

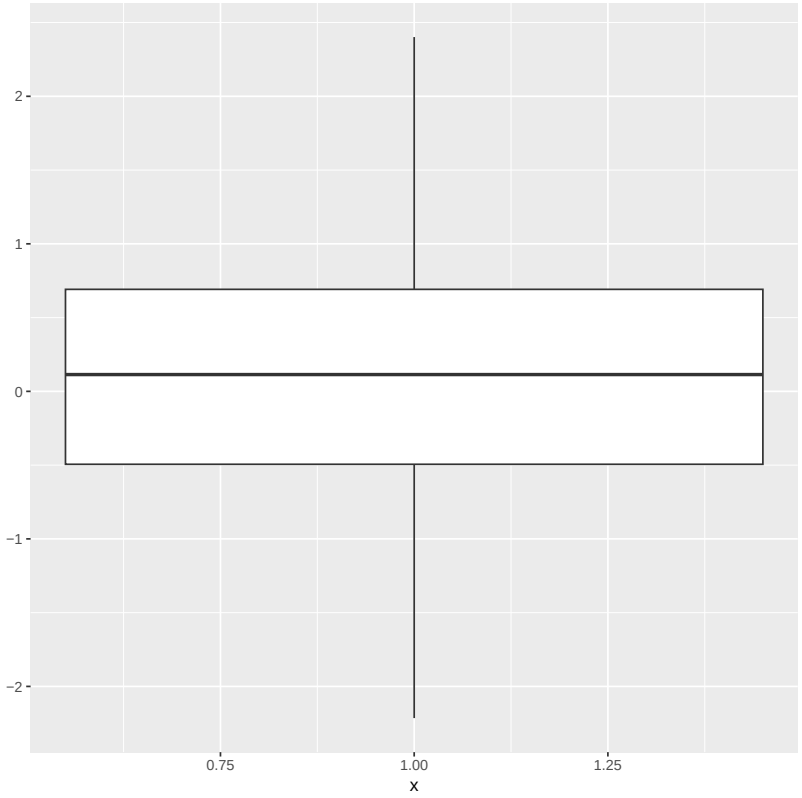



```
# Adjust the transparency of outliers using outlier.alpha
ggplot(diamonds, aes(carat, price)) +
  geom_boxplot(aes(group = cut_width(carat, 0.25)), outlier.alpha = 0.1)

## Warning:  Orientation is not uniquely specified when
both the x and y aesthetics are
## continuous.  Picking default orientation 'x'.
```




```
## No test:
# It's possible to draw a boxplot with your own computations if you
# use stat = "identity":
set.seed(1)
y <- rnorm(100)
df <- data.frame(
  x = 1,
  y0 = min(y),
  y25 = quantile(y, 0.25),
  y50 = median(y),
  y75 = quantile(y, 0.75),
  y100 = max(y)
)
ggplot(df, aes(x)) +
  geom_boxplot(
   aes(ymin = y0, lower = y25, middle = y50, upper = y75, ymax = y100),
   stat = "identity"
 )
```



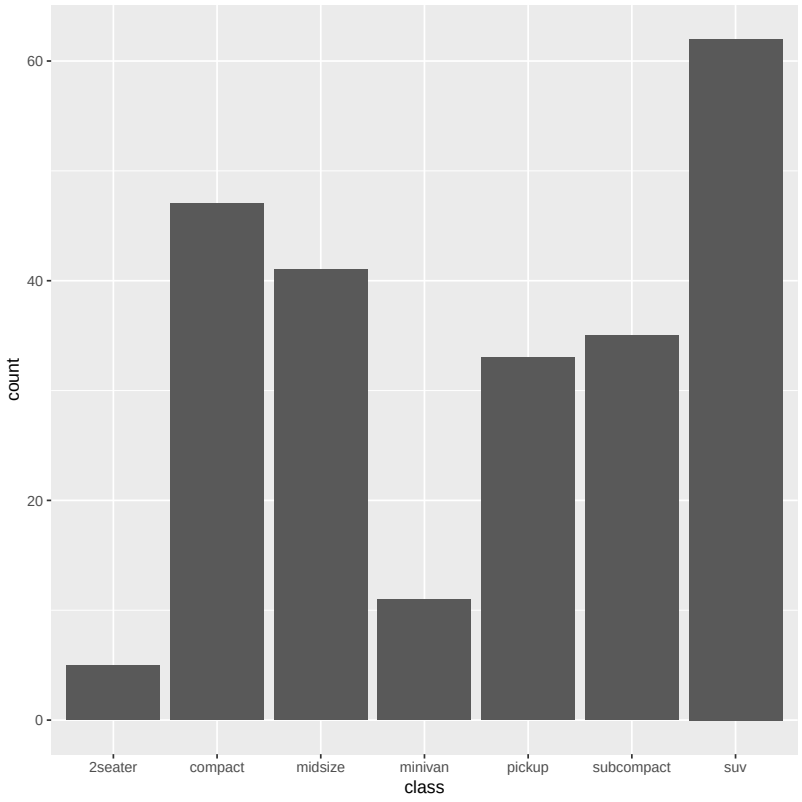

```
## End(No test)
```

geom_col

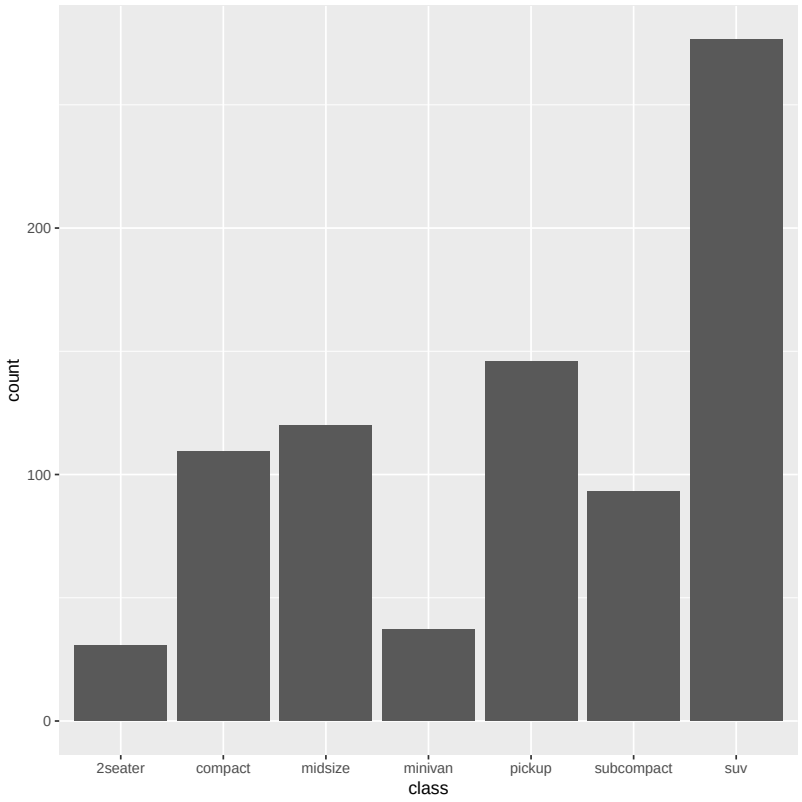
```
### Name: geom_bar
### Title: Bar charts
### Aliases: geom_bar geom_col stat_count

### ** Examples

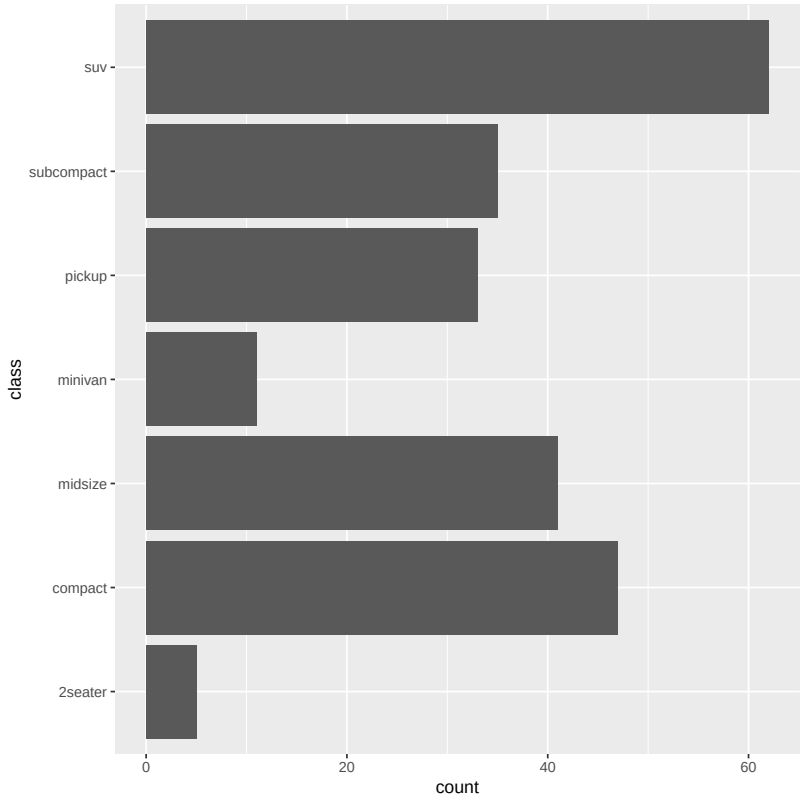
# geom_bar is designed to make it easy to create bar charts that show
# counts (or sums of weights)
g <- ggplot(mpg, aes(class))
# Number of cars in each class:
g + geom_bar()
```



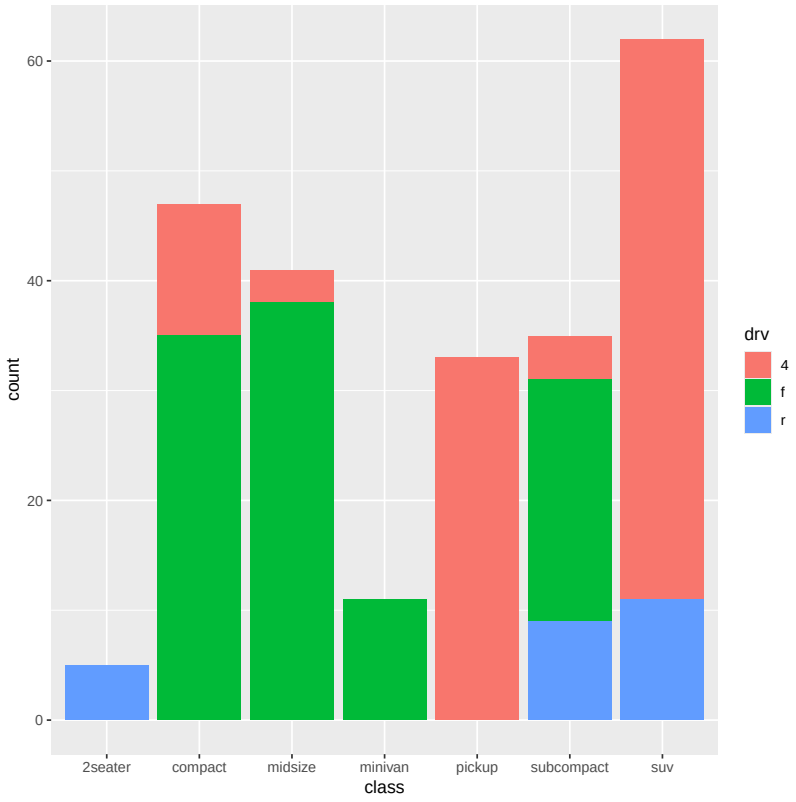

```
# Total engine displacement of each class
g + geom_bar(aes(weight = displ))
```



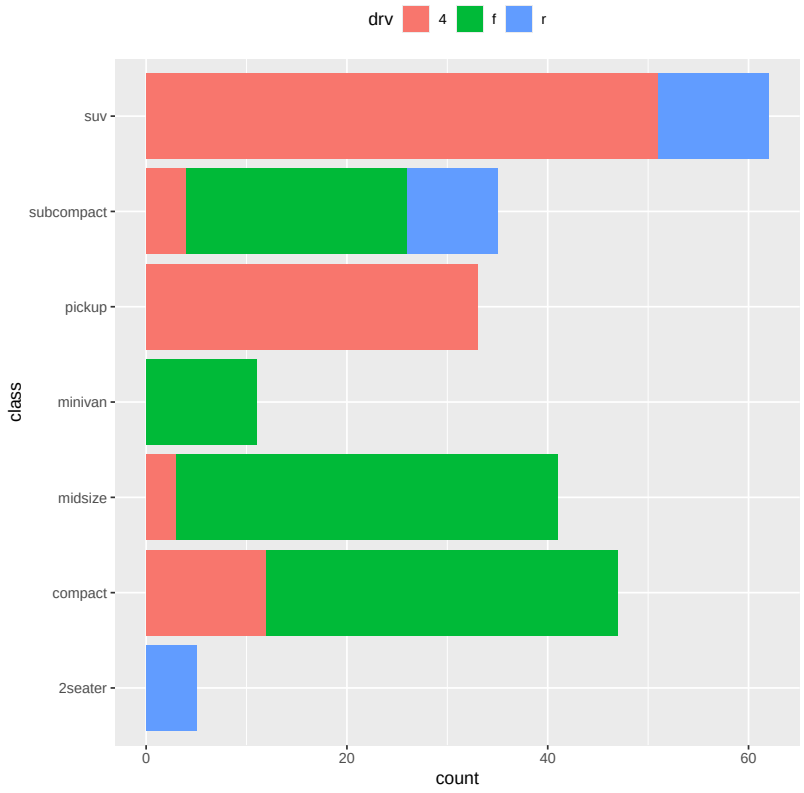
```
# Map class to y instead to flip the orientation
ggplot(mpg) + geom_bar(aes(y = class))
```



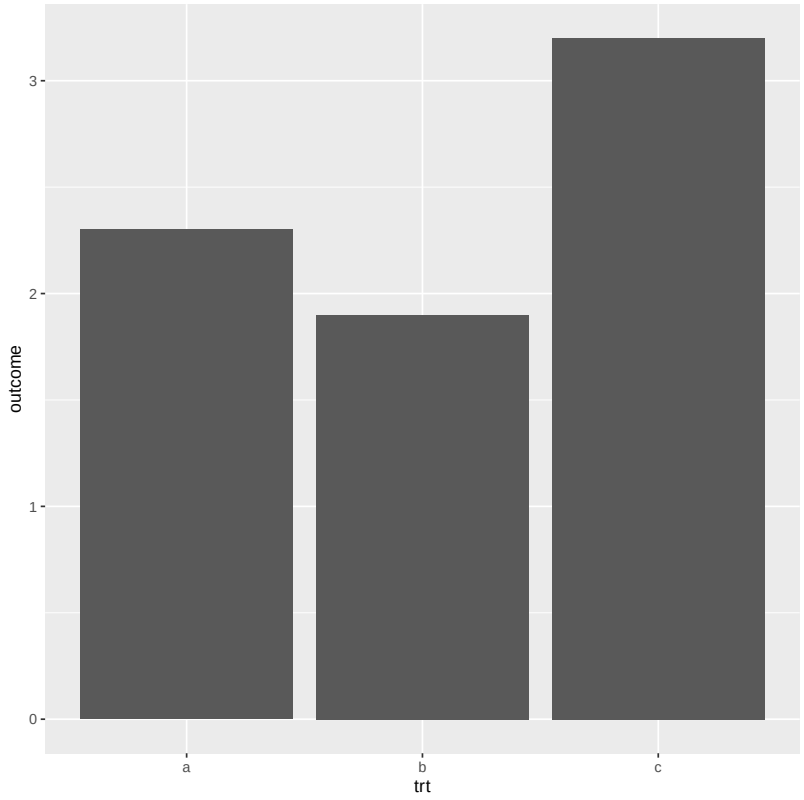

```
# Bar charts are automatically stacked when multiple bars are placed
# at the same location. The order of the fill is designed to match
# the legend
g + geom_bar(aes(fill = drv))
```



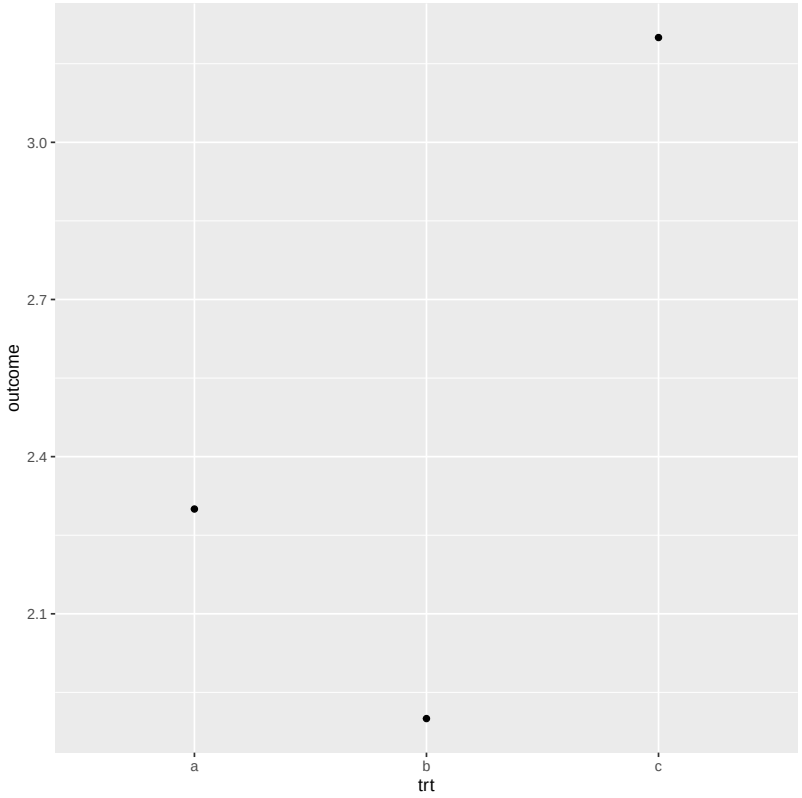

```
# If you need to flip the order (because you've flipped the orientation)
# call position_stack() explicitly:
ggplot(mpg, aes(y = class)) +
 geom_bar(aes(fill = drv), position = position_stack(reverse = TRUE)) +
 theme(legend.position = "top")
```



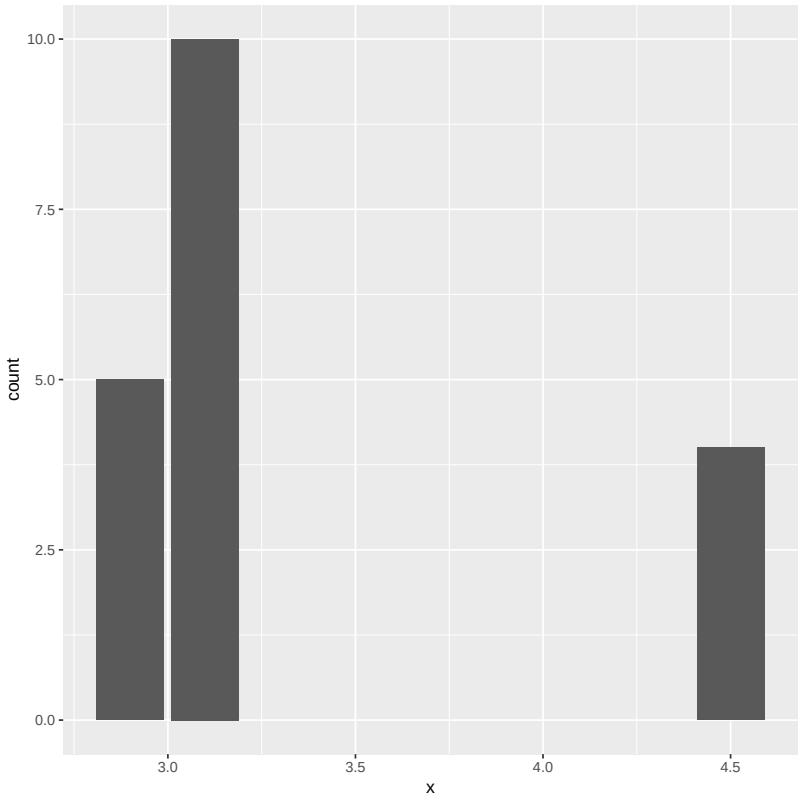

```r
# To show (e.g.) means, you need geom_col()
df <- data.frame(trt = c("a", "b", "c"), outcome = c(2.3, 1.9, 3.2))
ggplot(df, aes(trt, outcome)) +
  geom_col()
```



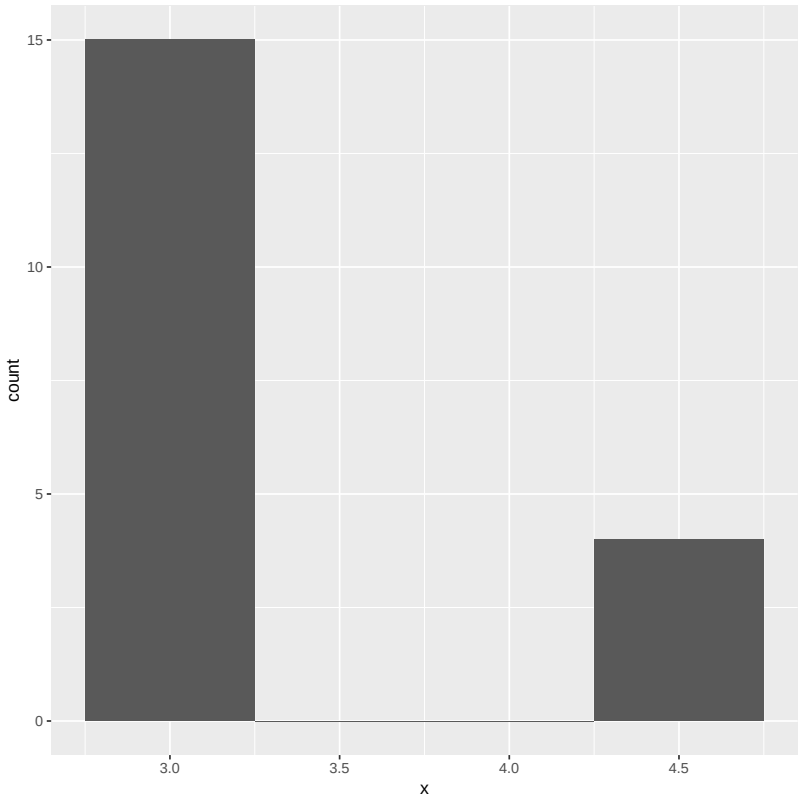
```
# But geom_point() displays exactly the same information and doesn't
# require the y-axis to touch zero.
ggplot(df, aes(trt, outcome)) +
  geom_point()
```



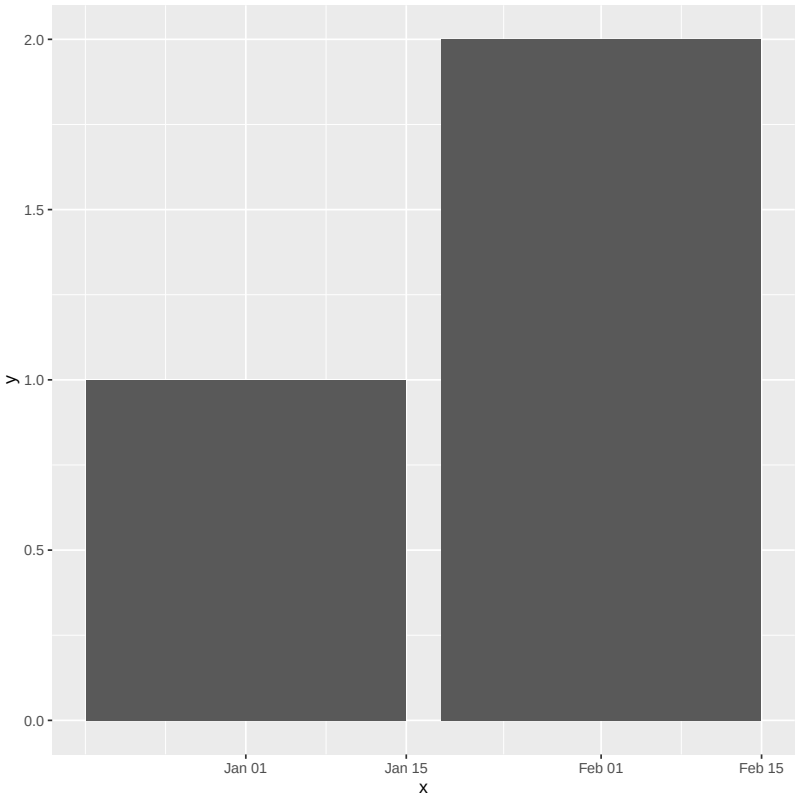

```
# You can also use geom_bar() with continuous data, in which case
# it will show counts at unique locations
df <- data.frame(x = rep(c(2.9, 3.1, 4.5), c(5, 10, 4)))
ggplot(df, aes(x)) + geom_bar()
```



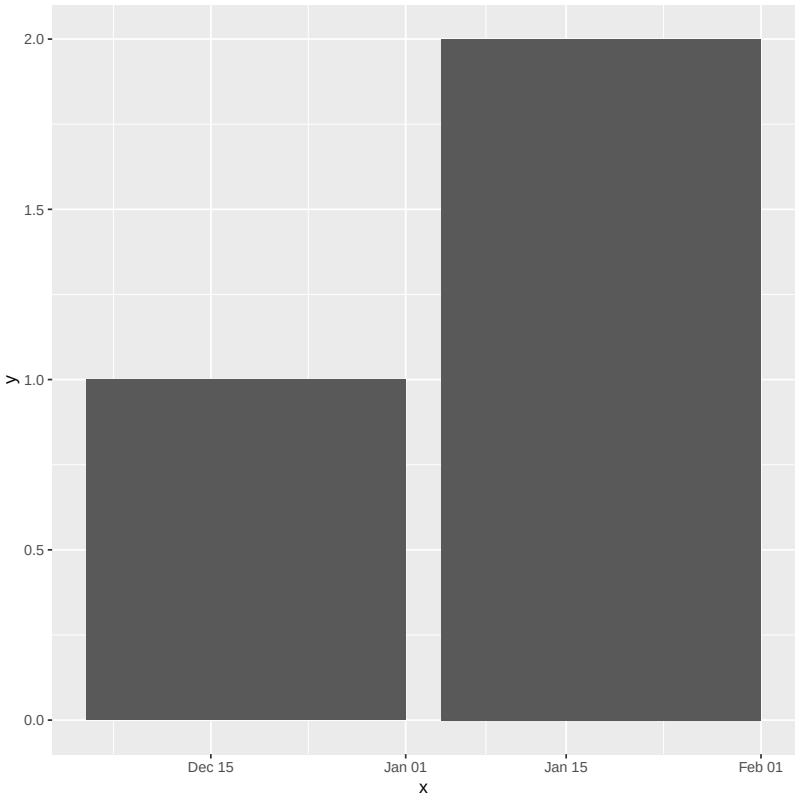

```
# cf. a histogram of the same data
ggplot(df, aes(x)) + geom_histogram(binwidth = 0.5)
```



```r
# Use `just` to control how columns are aligned with axis breaks:
df <- data.frame(x = as.Date(c("2020-01-01", "2020-02-01")), y = 1:2)
# Columns centered on the first day of the month
ggplot(df, aes(x, y)) + geom_col(just = 0.5)
```




```r
# Columns begin on the first day of the month
ggplot(df, aes(x, y)) + geom_col(just = 1)
```

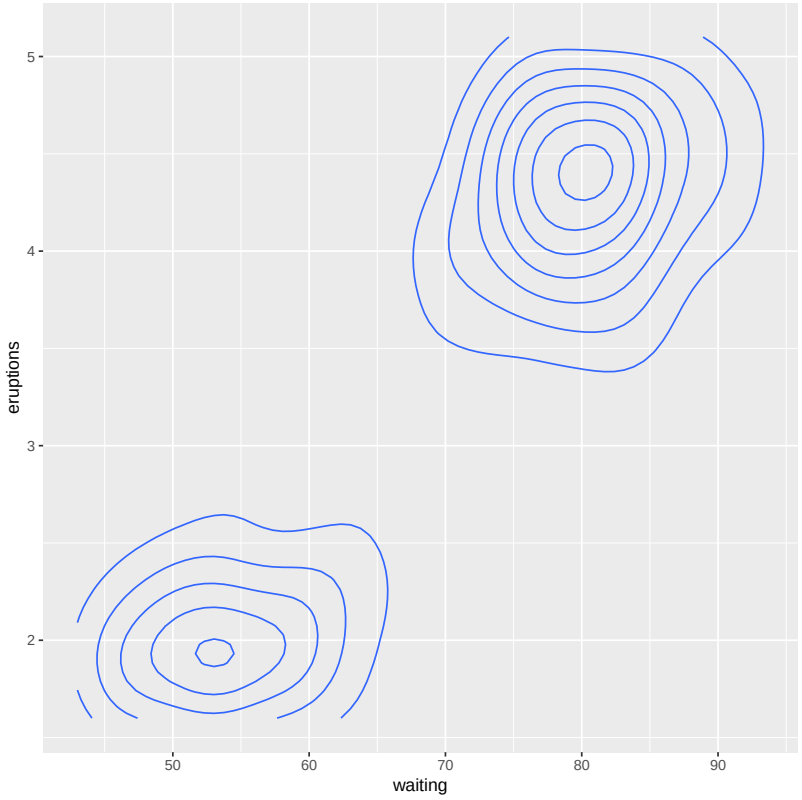


geom_contour

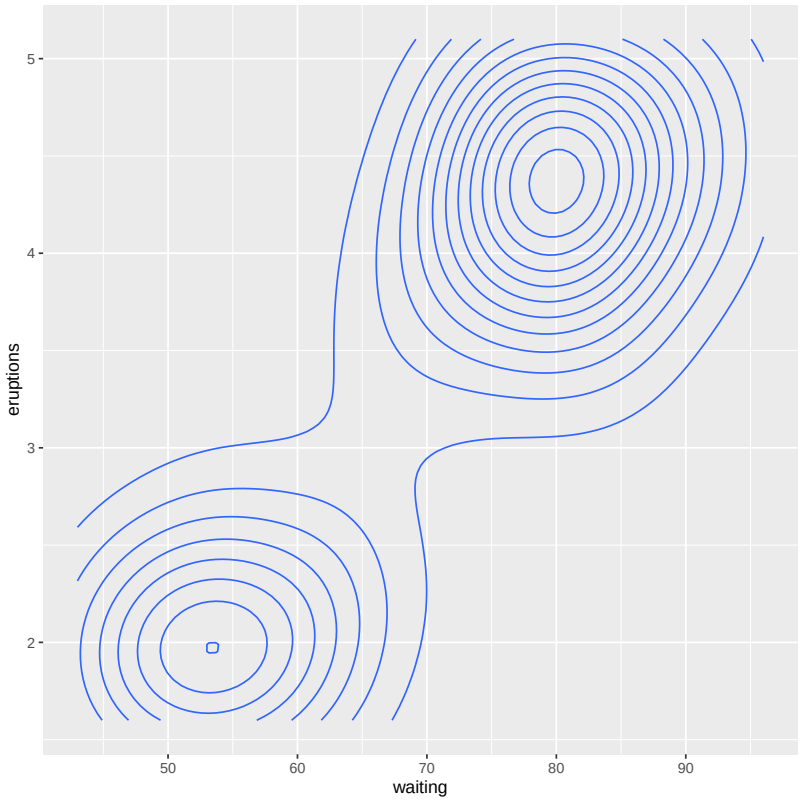
```
### Name: geom_contour
### Title: 2D contours of a 3D surface
### Aliases: geom_contour geom_contour_filled stat_contour
###   stat_contour_filled

### ** Examples

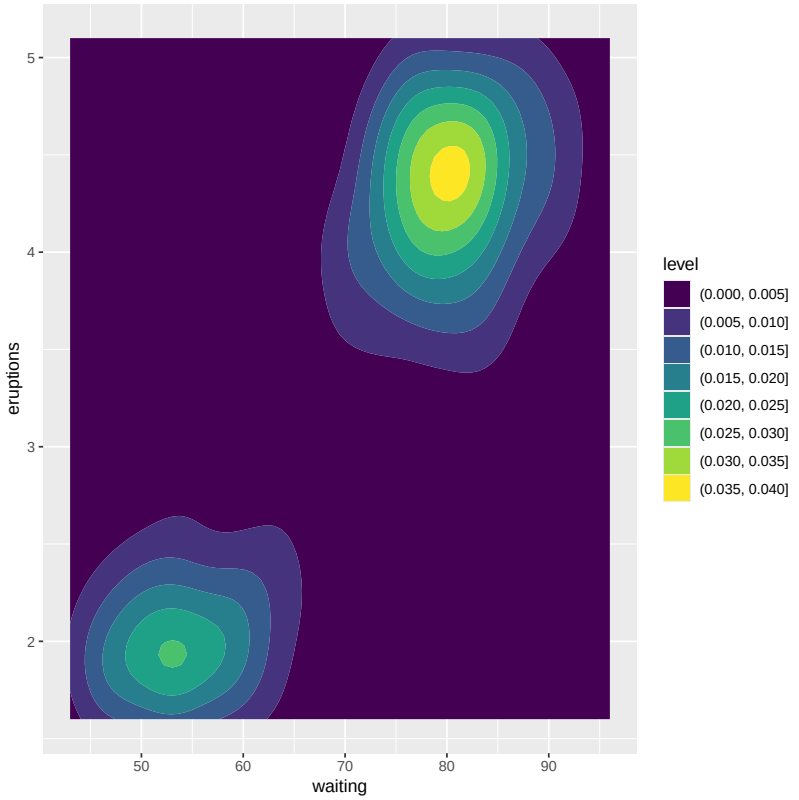
# Basic plot
v <- ggplot(faithfuld, aes(waiting, eruptions, z = density))
v + geom_contour()
```



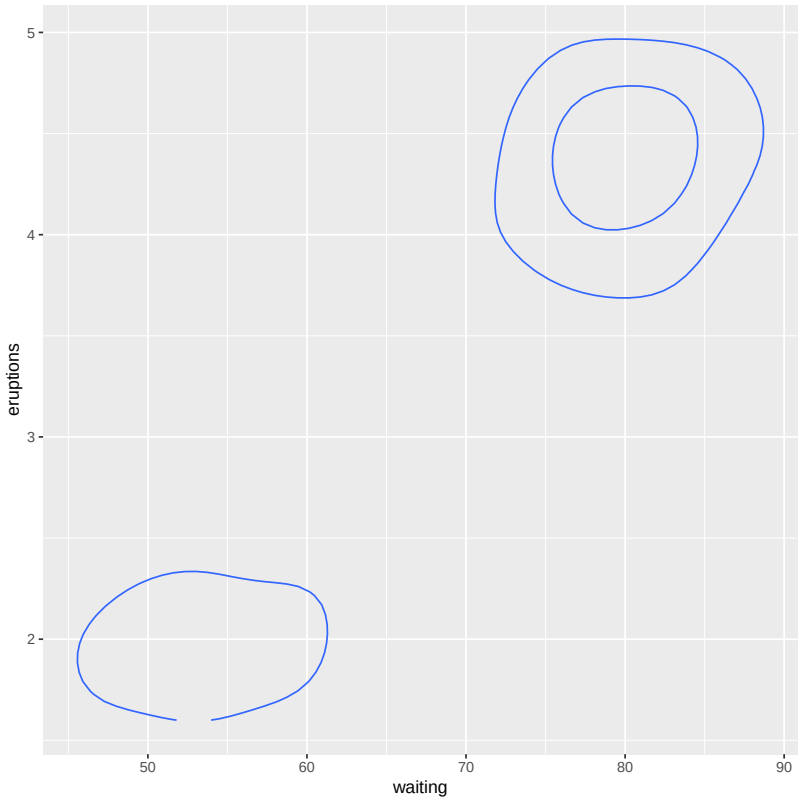

```
# Or compute from raw data
ggplot(faithful, aes(waiting, eruptions)) +
  geom_density_2d()
```



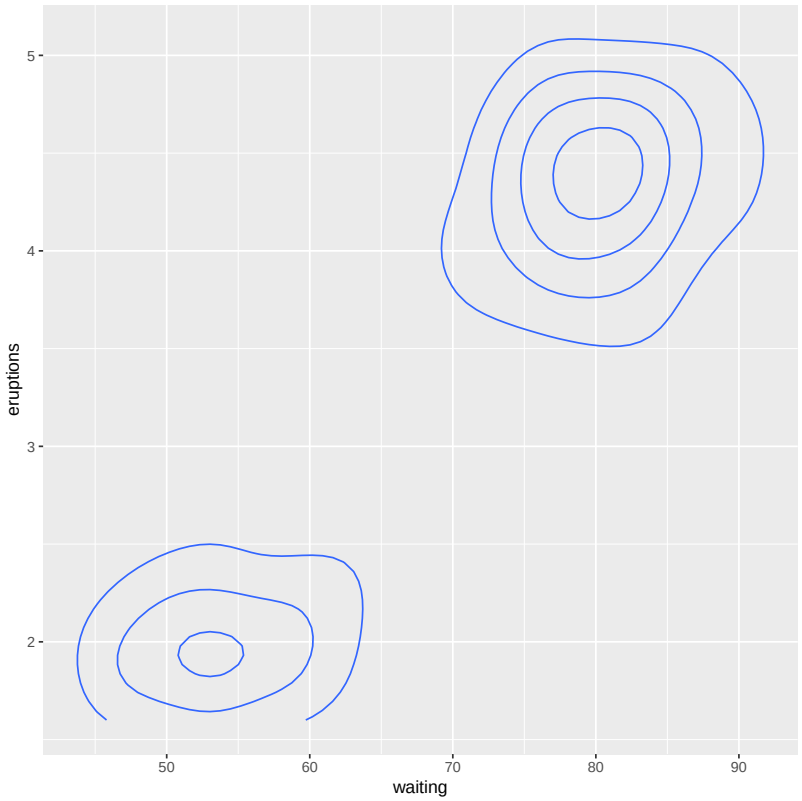
```
## No test:
# use geom_contour_filled() for filled contours
v + geom_contour_filled()
```



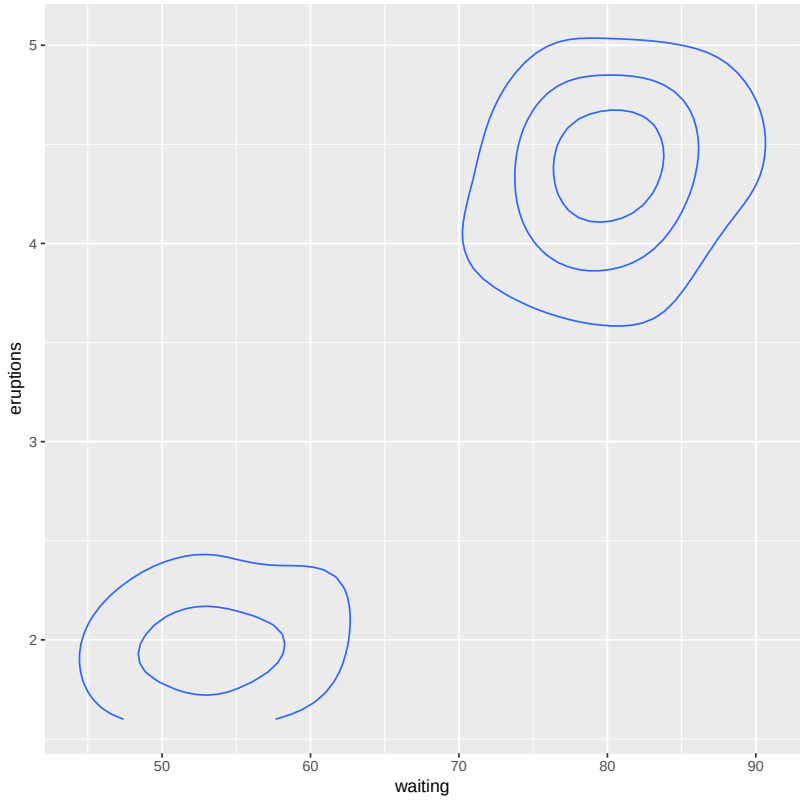

```
# Setting bins creates evenly spaced contours in the range of the data
v + geom_contour(bins = 3)
```



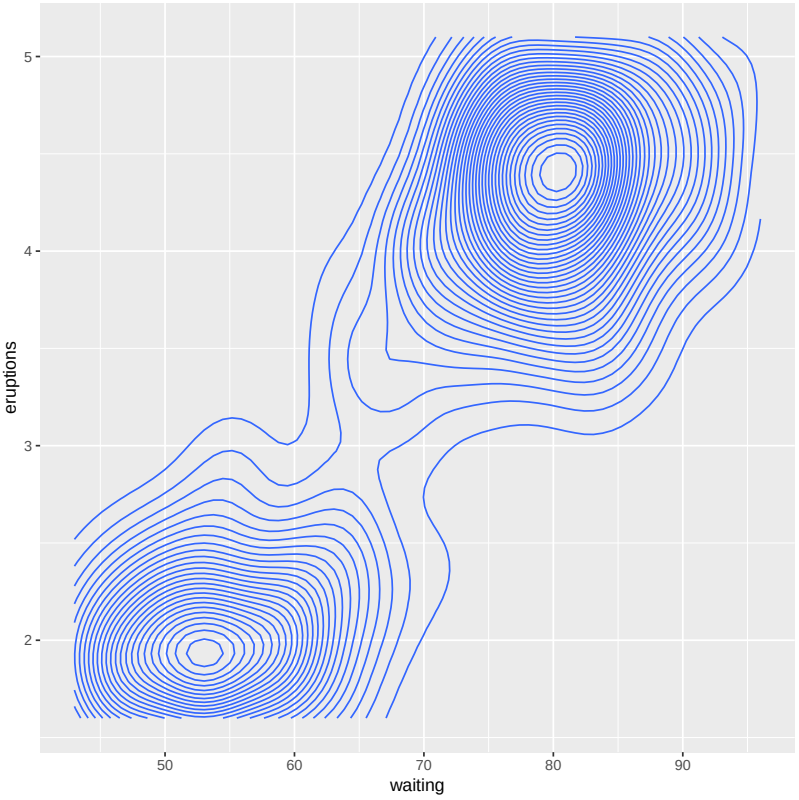

```
v + geom_contour(bins = 5)
```



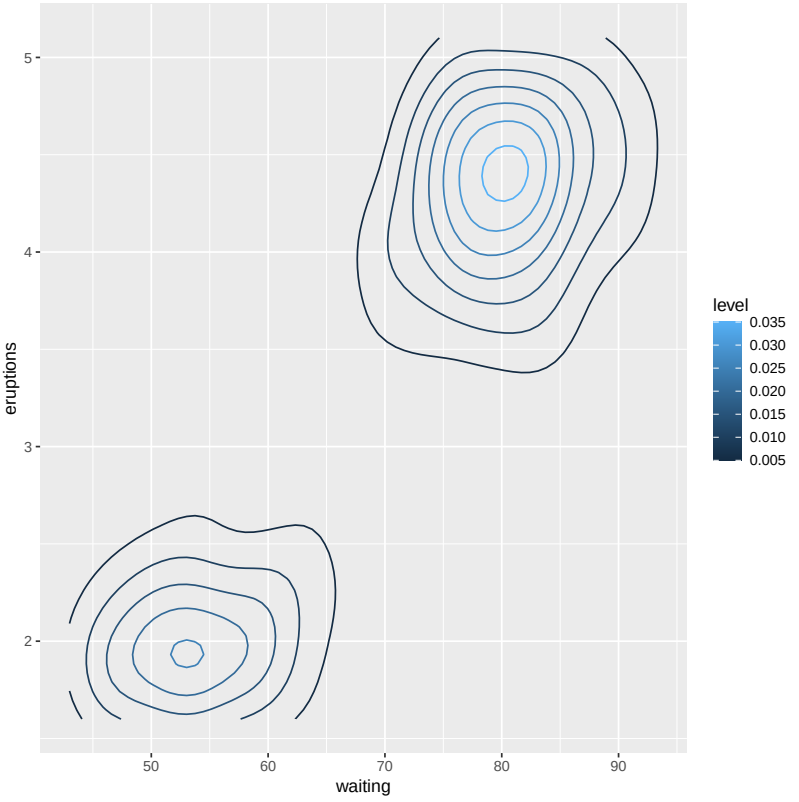
```
# Setting binwidth does the same thing, parameterised by the distance
# between contours
v + geom_contour(binwidth = 0.01)
```



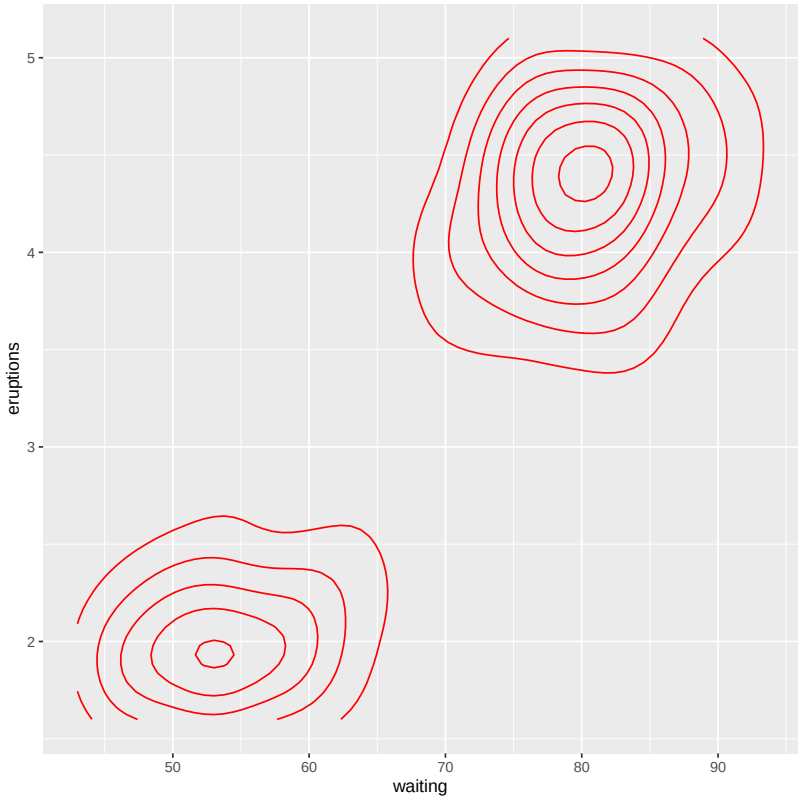

```
v + geom_contour(binwidth = 0.001)
```



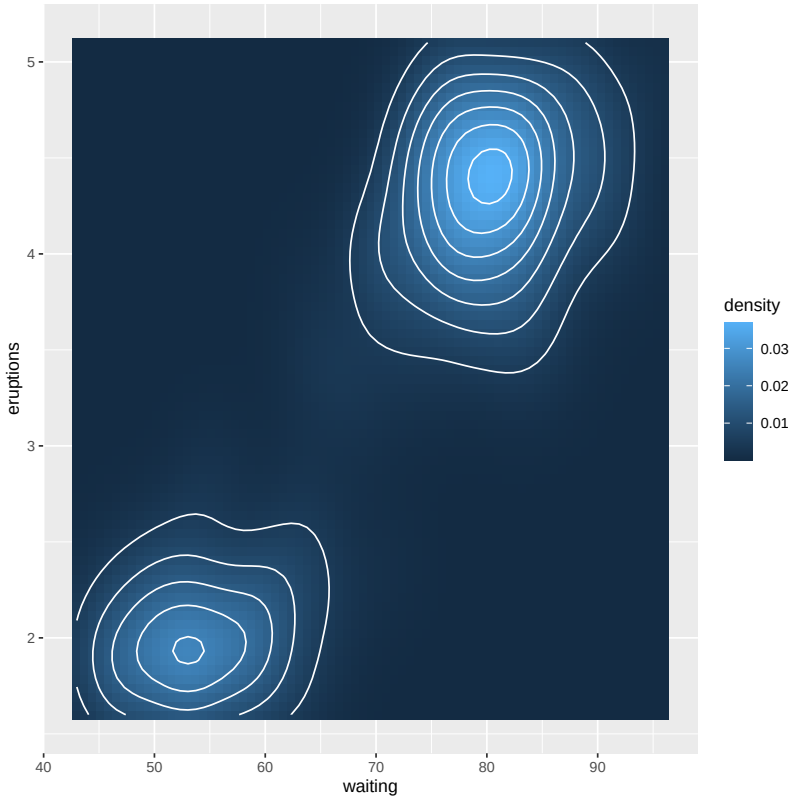

```
# Other parameters
v + geom_contour(aes(colour = after_stat(level)))
```



```
v + geom_contour(colour = "red")
```




```
v + geom_raster(aes(fill = density)) +
  geom_contour(colour = "white")
```



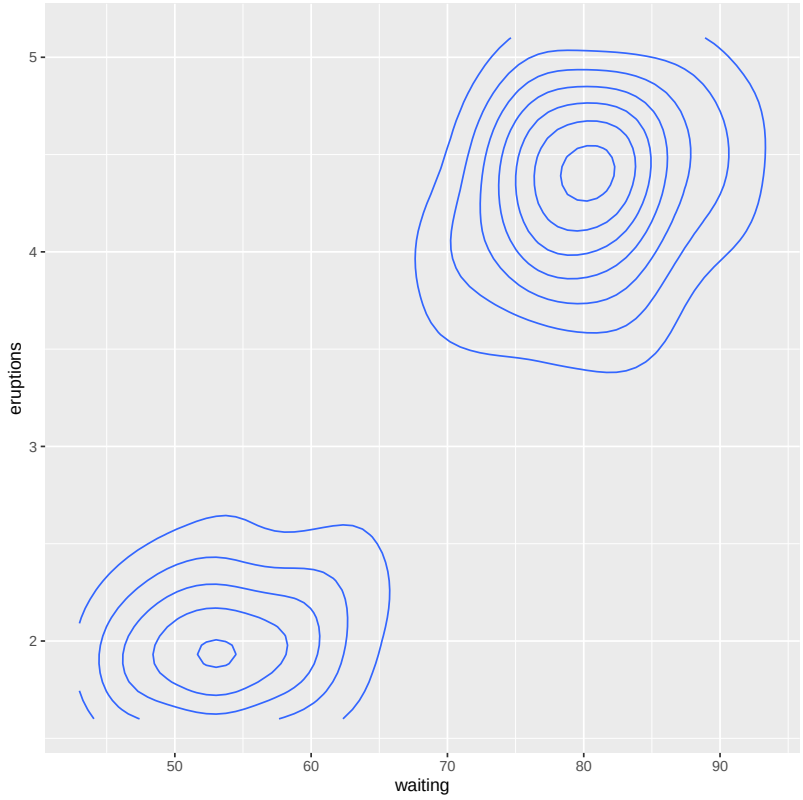
```
## End(No test)
```

geom_contour_filled

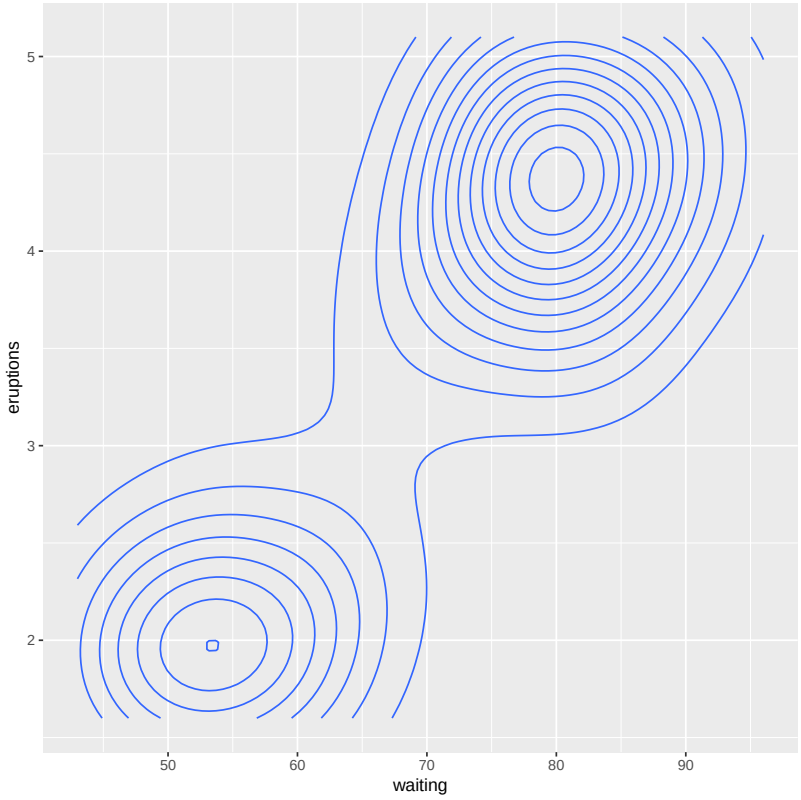
```
### Name: geom_contour
### Title: 2D contours of a 3D surface
### Aliases: geom_contour geom_contour_filled stat_contour
###   stat_contour_filled

### ** Examples

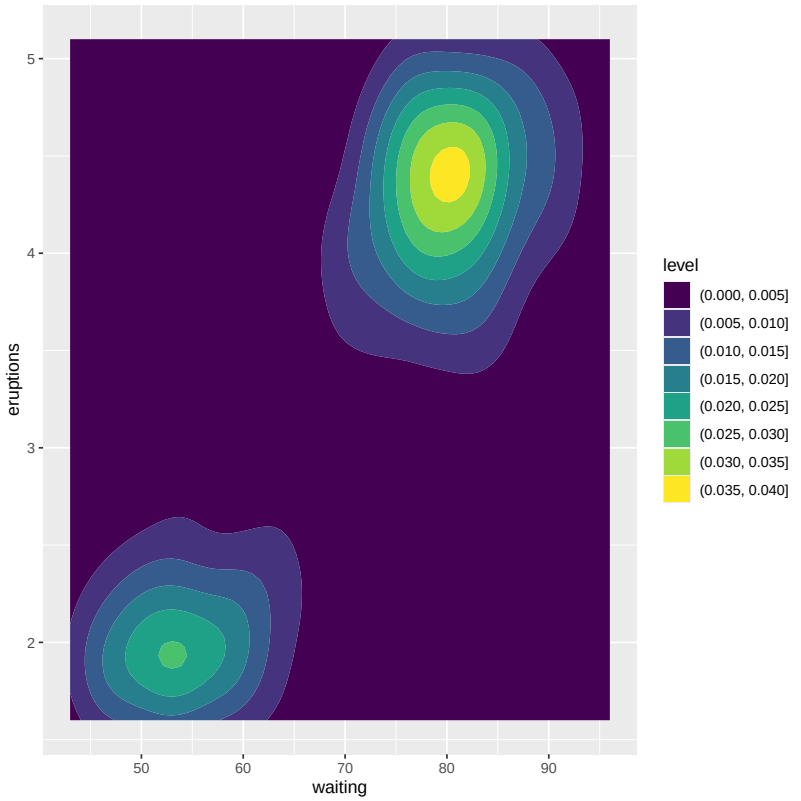
# Basic plot
v <- ggplot(faithfuld, aes(waiting, eruptions, z = density))
v + geom_contour()
```



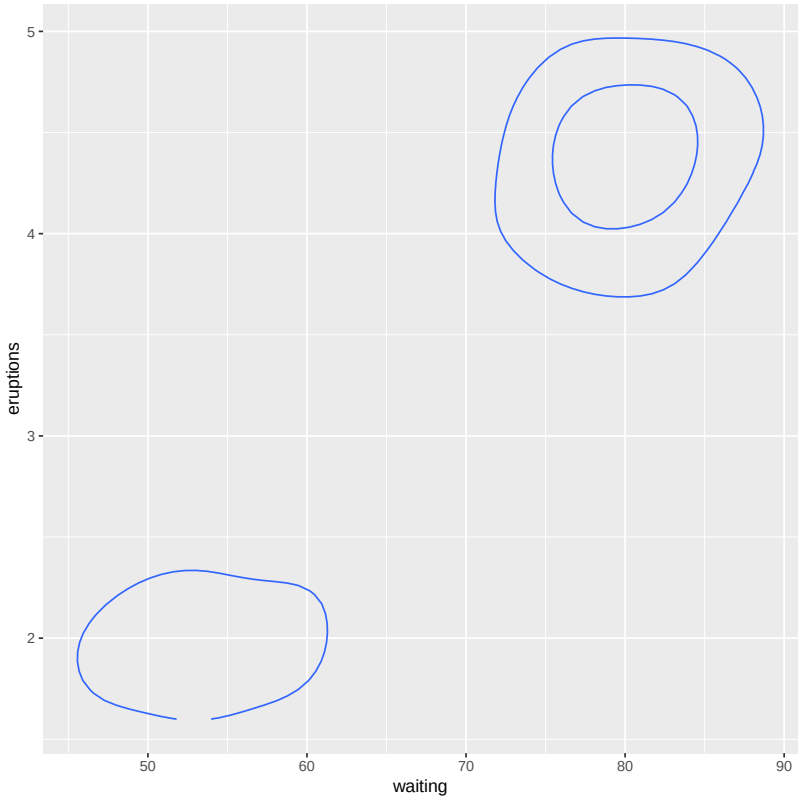

```
# Or compute from raw data
ggplot(faithful, aes(waiting, eruptions)) +
  geom_density_2d()
```



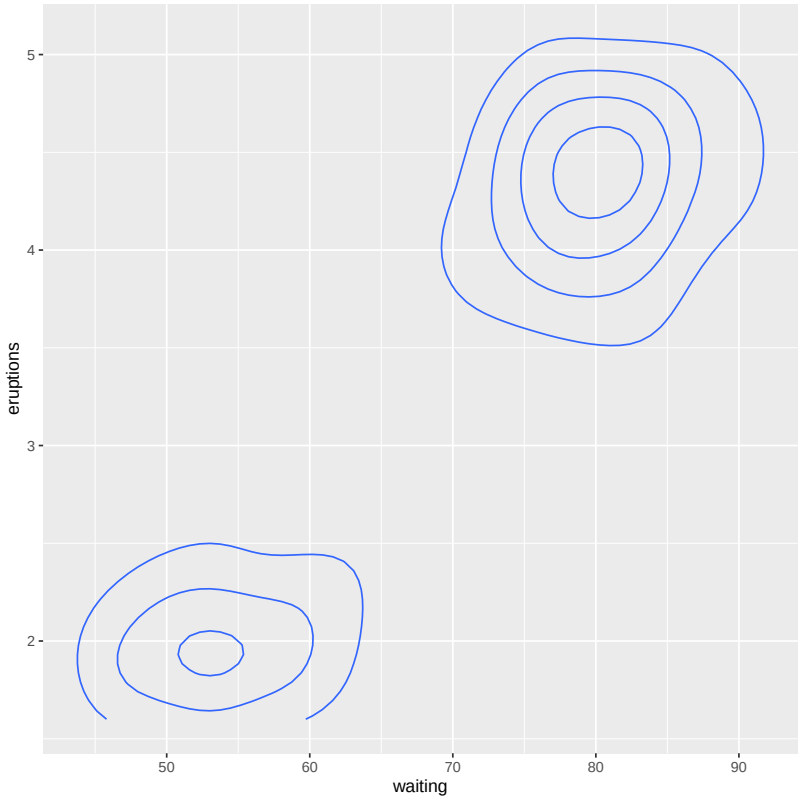

```
## No test:
# use geom_contour_filled() for filled contours
v + geom_contour_filled()
```



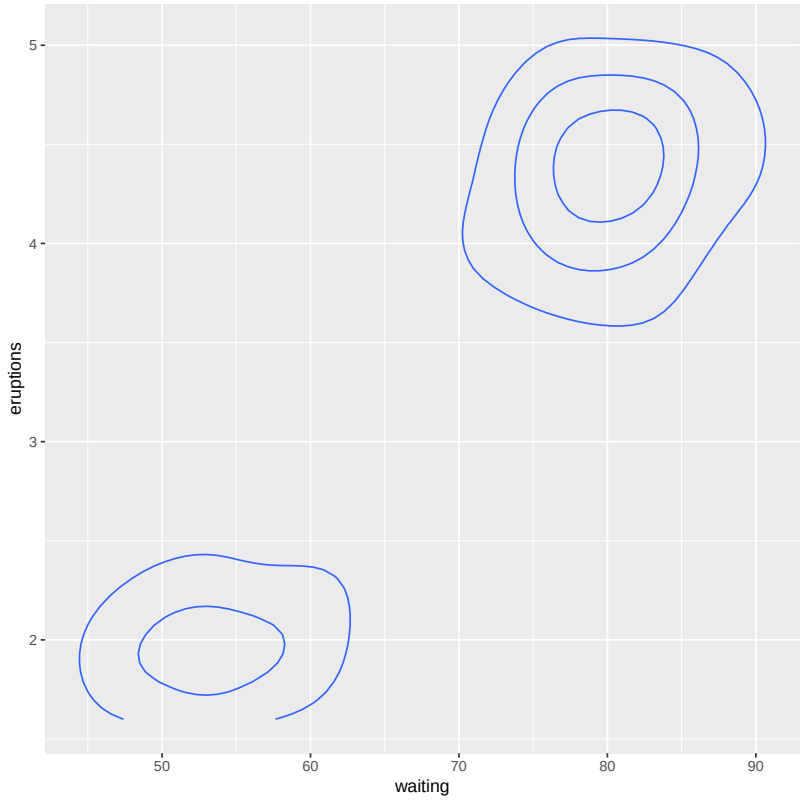
```
# Setting bins creates evenly spaced contours in the range of the data
v + geom_contour(bins = 3)
```



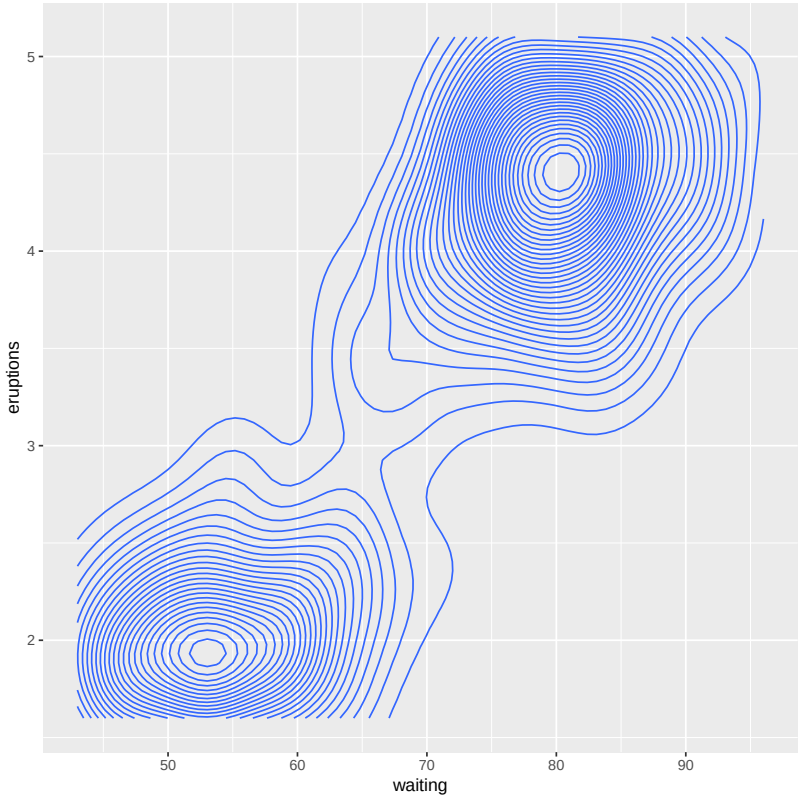

```
v + geom_contour(bins = 5)
```



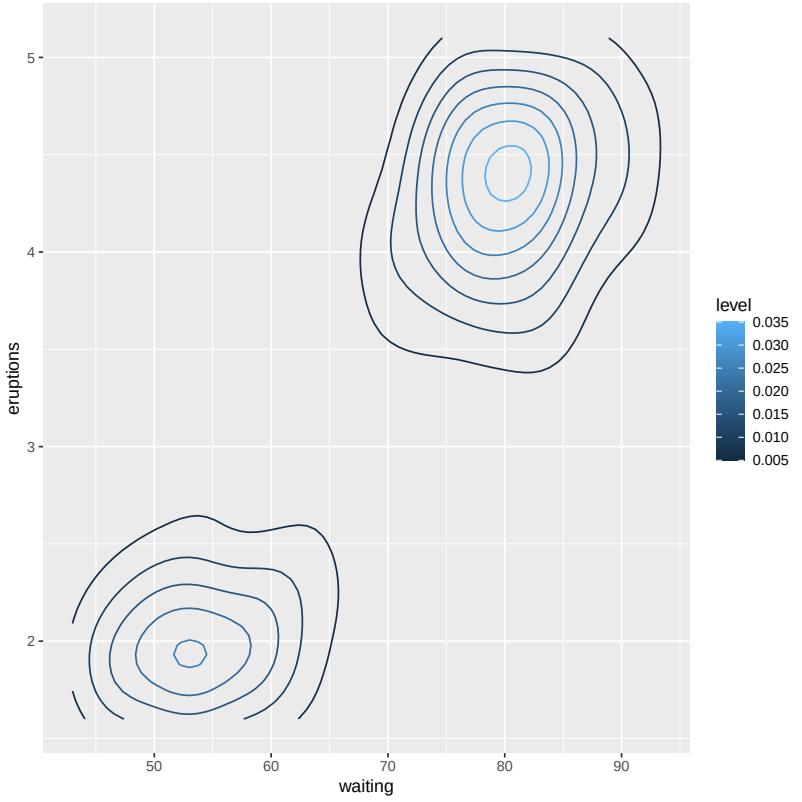
```
# Setting binwidth does the same thing, parameterised by the distance
# between contours
v + geom_contour(binwidth = 0.01)
```



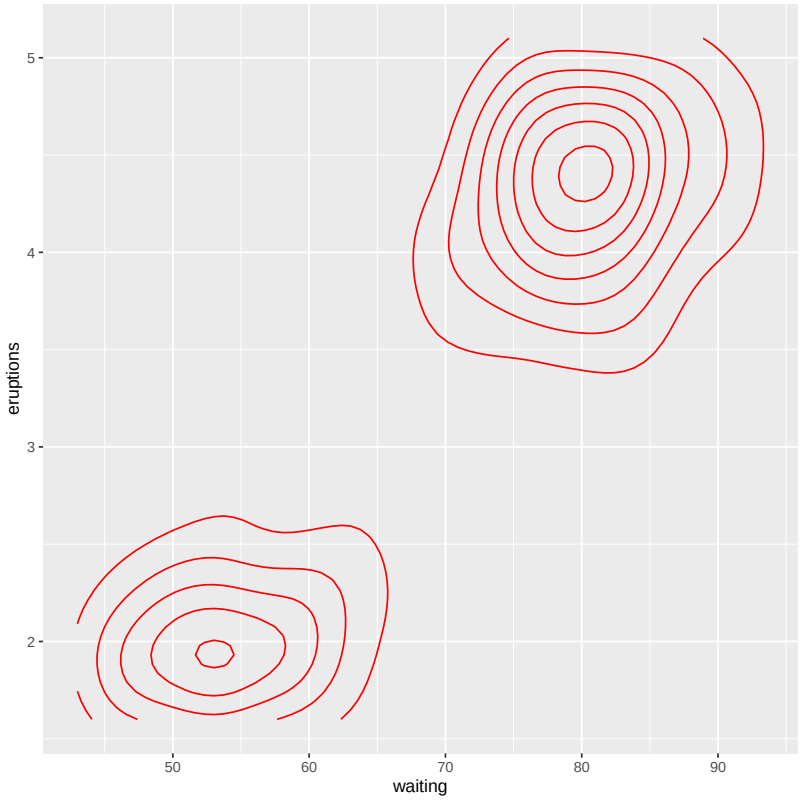

```
v + geom_contour(binwidth = 0.001)
```



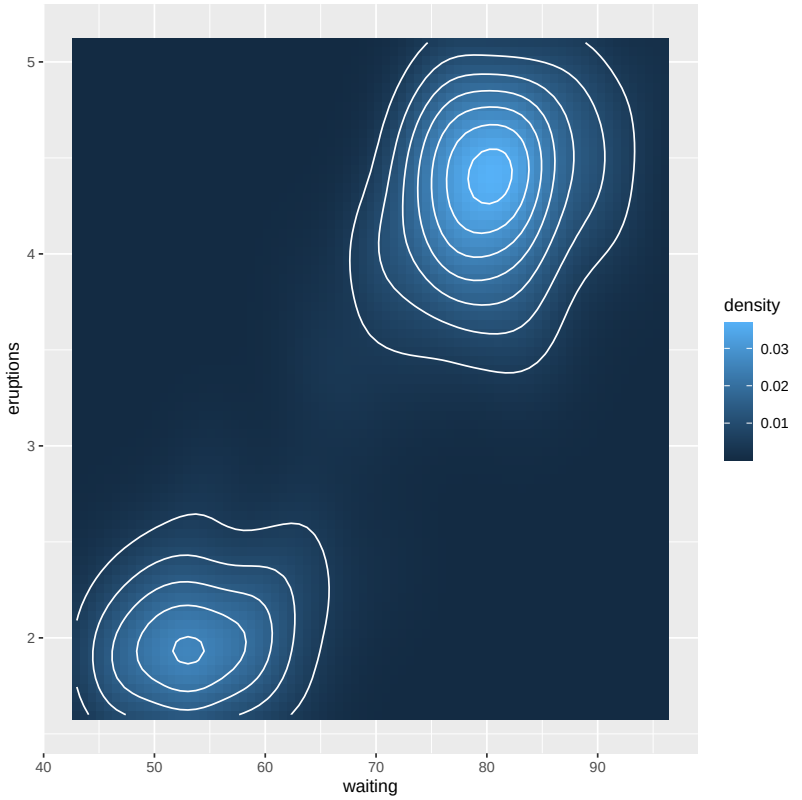

```
# Other parameters
v + geom_contour(aes(colour = after_stat(level)))
```



```
v + geom_contour(colour = "red")
```




```
v + geom_raster(aes(fill = density)) +
  geom_contour(colour = "white")
```



```
## End(No test)
```